

IAD V1 — Section 1

Repository Structure & Implementation Foundation

Document: Implementation Architecture Document (IAD V1)

Section: 1

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)

1.1 Purpose

This section defines the repository structure, implementation organization, source-code governance, and foundational development standards for TODP.

The repository structure exists to ensure:

- Maintainability

- Scalability
- Traceability
- Testability
- Reproducibility
- Team Collaboration

The structure shall align with:

- Domain-Driven Design (DDD V1)
- System Design (SDD V1)
- Architectural Principles
- Operational Governance Requirements

1.2 Repository Philosophy

The repository shall be organized around business domains rather than technologies.

BAD

frontend/

backend/

database/

GOOD

opportunities/

authorities/

evidence/

ranking/

recommendation/

creation/

The repository must reflect the business architecture.

1.3 Repository Objectives

OBJ-01

Support independent development.

OBJ-02

Support domain ownership.

OBJ-03

Support future service extraction.

OBJ-04

Support testing automation.

OBJ-05

Support deployment automation.

OBJ-06

Support governance requirements.

1.4 Monorepo Strategy

TODP V1 shall use a monorepo architecture.

Reasons

Shared domain models

Shared contracts

Shared testing

Simplified deployment

Simplified governance

Unified versioning

Future versions may evolve into multi-repository architecture.

1.5 Repository Root Structure

todp/

```
|— frontend/
|— backend/
|— database/
|— infrastructure/
|— observability/
|— docs/
|— scripts/
|— tests/
|— .github/
|— docker/
|— .env.example
|— docker-compose.yml
|— README.md
```

Purpose

frontend

React Application

backend

FastAPI Services

database

Database Assets

infrastructure

Infrastructure Definitions

observability

Telemetry Assets

docs

Architecture Documents

scripts

Automation Scripts

tests

Cross-System Tests

docker

Container Definitions

1.6 Frontend Structure

frontend/

├─ src/

├─ public/

├─ tests/

├─ package.json

├─ vite.config.ts

└─ README.md

1.7 Frontend Source Structure

frontend/src/

└─ app/
└─ pages/
└─ features/
└─ components/
└─ services/
└─ hooks/
└─ contexts/
└─ types/
└─ utils/
└─ assets/

Purpose

app/

Application bootstrap

pages/

Route-level screens

features/

Business domains

components/

Reusable UI components

services/

API integrations

hooks/

Custom React hooks

contexts/

Application state

types/

TypeScript models

utils/

Helper functions

assets/

Images and resources

1.8 Feature-Based Frontend Architecture

frontend/src/features/

|— discovery/

|— authority/

|— evidence/

|— ranking/

|— recommendation/

|— opportunity/

|— creation/

|— user/

Objective

Align UI implementation with business domains.

1.9 Backend Structure

backend/

```
└─ app/
└─ tests/
└─ migrations/
└─ requirements/
└─ scripts/
└─ Dockerfile
└─ README.md
```

1.10 Backend Application Structure

backend/app/

```
└─ api/
└─ domains/
└─ services/
└─ repositories/
└─ models/
└─ schemas/
└─ infrastructure/
└─ security/
└─ observability/
└─ governance/
└─ main.py
```

1.11 Domain Structure

backend/app/domains/

└─ opportunity/

└─ authority/

└─ evidence/

└─ ranking/

└─ recommendation/

└─ creation/

└─ user/

└─ audit/

Purpose

Each domain owns:

- Models
- Business Rules
- Services
- Validation Logic

1.12 API Structure

backend/app/api/

```
├─ v1/
|   ├── opportunity.py
|   ├── authority.py
|   ├── evidence.py
|   ├── ranking.py
|   ├── recommendation.py
|   ├── creation.py
|   └─ user.py
```

Purpose

Expose platform functionality.

1.13 Service Layer Structure

backend/app/services/

Responsibilities

Workflow orchestration

Cross-domain coordination

Transaction management

Business process execution

1.14 Repository Layer Structure

backend/app/repositories/

Responsibilities

Database access

Persistence operations

Query execution

Data retrieval

Business logic is prohibited in repositories.

1.15 Schema Layer Structure

backend/app/schemas/

Responsibilities

Request schemas

Response schemas

Validation models

DTOs

1.16 Infrastructure Layer

backend/app/infrastructure/

|— database/

|— cache/

|— messaging/

|— storage/

|— config/

Purpose

External system integration.

1.17 Security Layer

backend/app/security/

|— authentication/

|— authorization/

|— secrets/

|— encryption/

Purpose

Centralized security implementation.

1.18 Governance Layer

backend/app/governance/

├─ audit/

├─ lineage/

├─ compliance/

├─ traceability/

Purpose

Support trust architecture.

1.19 Database Structure

database/

|— migrations/

|— seed/

|— schema/

|— procedures/

|— views/

Purpose

Database lifecycle management.

1.20 Migration Strategy

Migration framework shall support:

Versioning

Rollback

Auditability

Repeatability

Migration naming example

001_initial_schema.sql

002_authority_tables.sql

003_evidence_tables.sql

1.21 Observability Structure

observability/

├─ grafana/

├─ prometheus/

├─ opentelemetry/

├─ dashboards/

├─ alerts/

Purpose

Operational visibility.

1.22 Infrastructure Structure

infrastructure/

├─ environments/

| └─ dev/

| └─ test/

| └─ stage/

| └─ prod/

├─ networking/

├─ database/

├─ monitoring/

└─ security/

Purpose

Infrastructure-as-Code assets.

1.23 Test Structure

tests/

- └─ unit/
- └─ integration/
- └─ contract/
- └─ e2e/
- └─ performance/
- └─ security/
- └─ governance/

Purpose

Support SDD testing architecture.

1.24 Documentation Structure

docs/

- └─ PRD/
- └─ ADD/
- └─ DDD/
- └─ SDD/
- └─ IAD/

|— ADR/

|— RUNBOOKS/

Purpose

Single source of truth.

1.25 Environment Files

Required

.env.example

.env.dev

.env.test

.env.stage

.env.prod

Secrets shall never be committed.

1.26 Configuration Governance

Configuration shall be:

Versioned

Reviewed

Audited

Traceable

1.27 Naming Standards

Directories

snake_case

Files

snake_case

Classes

PascalCase

Functions

snake_case

Database Tables

snake_case

API Routes

kebab-case

1.28 Repository Governance Rules

REP-01

Business domains own business logic.

REP-02

Infrastructure must remain isolated.

REP-03

Business rules must not exist in UI.

REP-04

Business rules must not exist in repositories.

REP-05

Domain ownership must remain explicit.

REP-06

Repository structure must align with DDD.

1.29 Architectural Constraints

AC-REP-01

No domain may directly access another domain database tables.

AC-REP-02

Cross-domain interaction must occur through services.

AC-REP-03

All code must remain testable.

AC-REP-04

All code must remain observable.

AC-REP-05

All code must remain auditable where required.

1.30 Section Summary

The TODP repository structure establishes the implementation foundation for the platform.

The structure aligns:

Business Domains

↓

Application Services



Persistence



Infrastructure



Governance



Operations

ensuring maintainability, scalability, traceability, auditability, and reproducibility throughout the implementation lifecycle.

IAD Progress

IAD V1 — Section 2

Frontend Implementation Architecture (React)

Document: Implementation Architecture Document (IAD V1)

Section: 2

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)

2.1 Purpose

This section defines the React frontend implementation architecture for TODP.

The frontend is responsible for:

- User interaction
- Opportunity discovery
- Authority visibility
- Evidence visibility
- Ranking visualization
- Recommendation presentation
- Opportunity creation support

The frontend shall remain presentation-focused and shall not own business rules.

2.2 Frontend Principles

FP-01

Frontend displays decisions.

FP-02

Backend owns decisions.

FP-03

Frontend never acts as authority.

FP-04

Frontend never performs ranking calculations.

FP-05

Frontend must remain traceable.

FP-06

Frontend must remain testable.

2.3 Technology Stack

Framework:

React

Language:

TypeScript

Routing:

React Router

HTTP Client:

Axios

State Management:

React Context + Custom Hooks

Forms:

React Hook Form

UI:

Component Driven Architecture

Build Tool:

Vite

Testing:

Vitest

React Testing Library

2.4 Frontend Architecture Overview

User



Pages



Features



Components



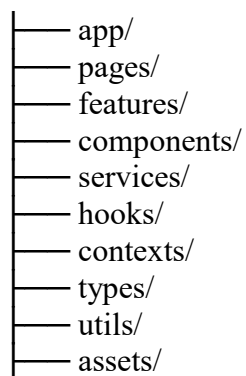
Services



Backend APIs

2.5 Source Structure

frontend/src/



2.6 Application Layer

frontend/src/app/

Responsibilities:

- App bootstrap
- Router initialization
- Global providers
- Error boundaries
- Theme configuration

Example:

App.tsx

router.tsx

providers.tsx

2.7 Page Layer

frontend/src/pages/

Pages represent route-level screens.

Example:

HomePage.tsx

DiscoveryPage.tsx

OpportunityPage.tsx

AuthorityPage.tsx

EvidencePage.tsx

RecommendationPage.tsx

CreationPage.tsx

ProfilePage.tsx

2.8 Feature Layer

frontend/src/features/

```
|— discovery/  
|— opportunity/  
|— authority/  
|— evidence/  
|— ranking/  
|— recommendation/  
|— creation/  
|— user/
```

Each feature owns:

- Components
- Hooks
- Services
- Types
- Tests

2.9 Discovery Feature

Purpose:

Opportunity search and exploration.

Structure:

features/discovery/

```
|— components/  
|— hooks/  
|— services/  
|— types/  
|— tests/
```

Responsibilities:

- Search
- Filters
- Retrieval display
- Result visualization

2.10 Opportunity Feature

Purpose:

Opportunity visualization.

Responsibilities:

- Opportunity details
 - Opportunity lifecycle state
 - Cost display
 - Eligibility display
-

2.11 Authority Feature

Purpose:

Authority transparency.

Responsibilities:

- Authority information
 - Authority verification status
 - Authority explanations
 - Authority lineage display
-

2.12 Evidence Feature

Purpose:

Evidence transparency.

Responsibilities:

- Evidence display
 - Evidence lineage
 - Evidence history
 - Evidence references
-

2.13 Ranking Feature

Purpose:

Ranking explanation.

Responsibilities:

- Cost breakdown
- Ranking position
- Ranking explanation
- Ranking transparency

Displayed:

Money Cost

Time Cost

Distance Cost

Total Cost

2.14 Recommendation Feature

Purpose:

Recommendation presentation.

Responsibilities:

- Recommendation display
 - Supporting evidence
 - Supporting authority
 - Ranking references
 - Explanation rendering
-

2.15 Creation Feature

Purpose:

Support opportunity creation.

Responsibilities:

- Similarity display
- Pattern display
- Gap display

- Opportunity concept display

Human remains final decision-maker.

2.16 Shared Component Layer

frontend/src/components/

Reusable components only.

Examples:

Button

Card

Modal

Table

Pagination

SearchBar

ErrorBoundary

Loader

EmptyState

2.17 Component Rules

CMP-01

Components must be reusable.

CMP-02

Components must not contain business logic.

CMP-03

Components must remain stateless where possible.

CMP-04

Components must remain independently testable.

2.18 Service Layer

frontend/src/services/

Responsibilities:

- API communication
- Request execution
- Response handling
- Error handling

Examples:

opportunity_service.ts

authority_service.ts

evidence_service.ts

ranking_service.ts

recommendation_service.ts

2.19 Service Rules

SER-01

Services call APIs.

SER-02

Services do not perform business calculations.

SER-03

Services do not store business state.

SER-04

Services remain replaceable.

2.20 State Management

State is divided into:

Global State

Feature State

Component State

Global State

User Session

Theme

Application Configuration

Feature State

Search Results

Filters

Recommendation Views

Component State

Form Inputs

Modal Visibility

UI Controls

2.21 Context Architecture

frontend/src/contexts/

Examples:

AuthContext

UserContext

ThemeContext

AppContext

Purpose:

Shared application state.

2.22 Hook Architecture

frontend/src/hooks/

Examples:

useAuth

useDiscovery

useOpportunity

useRecommendation

useAuthority

useEvidence

Purpose:

Reusable frontend behavior.

2.23 Routing Architecture

Example Routes

/

/discover

/opportunities

/opportunities/:id

/authorities

/evidence

/recommendations

/create

/profile

2.24 API Integration Architecture

Frontend



Axios Client



API Gateway



FastAPI Services

All backend communication occurs through API services.

2.25 Error Handling Architecture

Error Categories:

Validation Error

Authorization Error

Network Error

System Error

Unexpected Error

Users must receive understandable messages.

2.26 Loading State Architecture

Supported States:

Loading

Success

Empty

Error

Every page must support all states.

2.27 Security Requirements

Frontend must support:

Authentication

Authorization

Session Management

Input Validation

Secure Token Handling

Frontend never stores secrets.

2.28 Accessibility Requirements

Accessibility support:

Keyboard Navigation

Screen Reader Support

Semantic HTML

Focus Management

Accessible Forms

2.29 Observability Requirements

Frontend telemetry shall include:

Page Views

Search Events

Recommendation Events

Error Events

Performance Events

Integrated through OpenTelemetry.

2.30 Performance Requirements

Initial Load:

< 3 Seconds

Route Navigation:

< 500 ms

Search UI Response:
< 200 ms

Recommendation Display:
< 500 ms

2.31 Testing Architecture

Unit Tests

Component Tests

Feature Tests

Integration Tests

E2E Tests

Accessibility Tests

2.32 Frontend Governance Rules

FE-01

Frontend displays business outcomes.

FE-02

Frontend never owns business rules.

FE-03

Frontend never performs ranking calculations.

FE-04

Frontend never validates authorities.

FE-05

Frontend never validates evidence.

FE-06

Frontend remains traceable and testable.

2.33 Architectural Constraints

AC-FE-01

Business logic belongs to backend.

AC-FE-02

Ranking logic belongs to backend.

AC-FE-03

Authority validation belongs to backend.

AC-FE-04

Evidence validation belongs to backend.

AC-FE-05

Frontend must remain stateless where possible.

AC-FE-06

Frontend must remain domain-aligned.

2.34 Section Summary

The TODP Frontend Architecture provides a domain-driven React implementation aligned with the platform's trust model.

The architecture follows:

Pages

↓

Features

↓

Components



Services



Backend APIs

ensuring scalability, maintainability, traceability, auditability, and alignment with TODP's core principles.

IAD Progress

- ☐ Section 1 — Repository Structure & Implementation Foundation
- ☐ Section 2 — Frontend Implementation Architecture (React)
- ☐ Section 3 — Backend Implementation Architecture (FastAPI)

IAD V1 — Section 3

Backend Implementation Architecture (FastAPI)

Document: Implementation Architecture Document (IAD V1)

Section: 3

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)

3.1 Purpose

This section defines the FastAPI backend implementation architecture for TODP.

The backend is responsible for:

- Business rules
- Domain workflows
- Authority validation
- Evidence validation
- Ranking execution
- Recommendation generation
- Audit recording
- Traceability generation
- Governance enforcement

The backend is the authoritative execution layer of TODP.

3.2 Backend Principles

BE-01

Backend owns business logic.

BE-02

Backend owns ranking logic.

BE-03

Backend owns recommendation logic.

BE-04

Backend owns governance enforcement.

BE-05

Backend owns audit generation.

BE-06

Backend owns traceability generation.

3.3 Technology Stack

Framework:

FastAPI

Language:

Python

Validation:

Pydantic

ORM:
SQLAlchemy

Database Driver:
psycopg

Migrations:
Alembic

Authentication:
JWT

Caching:
Redis

Observability:
OpenTelemetry

Testing:
Pytest

3.4 Backend Architecture Overview

API Layer

↓

Application Layer

↓

Domain Layer

↓

Repository Layer

↓

Database Layer

3.5 Backend Structure

backend/app/

- api/
- domains/
- services/
- repositories/
- schemas/
- models/
- infrastructure/
- security/
- observability/
- governance/
- main.py

3.6 Layer Responsibilities

API Layer

Request entry point.

Application Layer

Workflow orchestration.

Domain Layer

Business rules.

Repository Layer

Persistence operations.

Infrastructure Layer

External integrations.

Governance Layer

Audit and traceability.

3.7 Main Application

main.py responsibilities:

- FastAPI bootstrap
- Middleware registration
- Route registration
- OpenTelemetry initialization
- Exception handlers
- Health endpoints

3.8 API Layer

backend/app/api/v1/

```
|— discovery.py
|— opportunity.py
|— authority.py
|— evidence.py
|— ranking.py
|— recommendation.py
|— creation.py
|— user.py
```

Responsibilities:

- Request validation
- Response serialization
- Authentication checks
- Authorization checks

No business logic allowed.

3.9 Domain Architecture

backend/app/domains/

```
|— opportunity/
|— authority/
```

— evidence/
— ranking/
— recommendation/
— creation/
— user/
— audit/

Each domain owns:

- Entities
- Rules
- Services
- Validation

3.10 Opportunity Domain

Responsibilities:

- Opportunity lifecycle
- Eligibility logic
- Opportunity status management
- Opportunity retrieval

Owns:

OpportunityEntity

OpportunityService

OpportunityRules

3.11 Authority Domain

Responsibilities:

- Authority registration
- Authority verification
- Authority confidence management
- Authority lifecycle

Owns:

AuthorityEntity

AuthorityService

AuthorityValidator

3.12 Evidence Domain

Responsibilities:

- Evidence registration
- Evidence validation
- Evidence lineage
- Evidence lifecycle

Owns:

EvidenceEntity

EvidenceService

EvidenceValidator

3.13 Ranking Domain

Responsibilities:

- Cost calculation
- Ranking generation
- Tie resolution
- Ranking explanation

Owns:

RankingEngine

CostCalculator

RankingExplainer

3.14 Ranking Formula

Total Cost

=

Money Cost

-

Time Cost

-

Distance Cost

Ranking Service is the only location where ranking calculations occur.

3.15 Recommendation Domain

Responsibilities:

- Recommendation assembly
- Recommendation explanation
- Evidence attachment
- Authority attachment

Recommendations never bypass rankings.

3.16 Creation Domain

Responsibilities:

- Similarity analysis
- Pattern analysis
- Gap analysis
- Opportunity concept generation

Final novelty decisions remain human-controlled.

3.17 Application Service Layer

backend/app/services/

Purpose:

Cross-domain orchestration.

Example:

Search Workflow

↓

Retrieval

↓

Authority

↓

Evidence

↓

Ranking

↓

Recommendation

3.18 Service Rules

APP-01

Can coordinate domains.

APP-02

Can manage transactions.

APP-03

Cannot bypass domain rules.

APP-04

Cannot bypass governance controls.

3.19 Repository Layer

backend/app/repositories/

Examples:

OpportunityRepository

AuthorityRepository

EvidenceRepository

RankingRepository

RecommendationRepository

Responsibilities:

CRUD operations

Queries

Persistence

3.20 Repository Rules

REP-01

No business logic.

REP-02

No ranking logic.

REP-03

No recommendation logic.

REP-04

Database access only.

3.21 Schema Layer

backend/app/schemas/

Responsibilities:

Request Models

Response Models

DTOs

Validation Models

Examples:

OpportunityCreateRequest

OpportunityResponse

AuthorityResponse

RecommendationResponse

3.22 Database Models

backend/app/models/

Responsibilities:

ORM mappings.

Database definitions.

Relationships.

Indexes.

Constraints.

3.23 Infrastructure Layer

backend/app/infrastructure/

- database/
- cache/
- messaging/
- storage/
- config/

Purpose:

External system access.

3.24 Cache Architecture

Redis responsibilities:

Search caching

Ranking caching

Recommendation caching

Reference caching

Cache never becomes source of truth.

3.25 Security Layer

backend/app/security/

- auth/
- permissions/
- jwt/
- encryption/

Responsibilities:

Authentication

Authorization

Token validation

Access control

3.26 Governance Layer

backend/app/governance/

- audit/
- lineage/
- traceability/
- compliance/

Responsibilities:

Audit generation

Lineage generation

Decision trace generation

Governance enforcement

3.27 Audit Integration

Every critical workflow generates:

Audit Record

↓

Trace Record

↓

Lineage Record

↓

Correlation ID

Mandatory for:

Authority workflows

Evidence workflows

Ranking workflows

Recommendation workflows

3.28 Observability Layer

backend/app/observability/

Responsibilities:

Metrics

Tracing

Logging

Telemetry

OpenTelemetry integrated at service boundaries.

3.29 Exception Architecture

Exception Categories:

ValidationException

AuthorizationException

BusinessRuleException

GovernanceException

InfrastructureException

All exceptions return standardized responses.

3.30 Response Standard

Every API response contains:

success

timestamp

correlation_id

data

errors

Example

```
{  
  "success": true,  
  "correlation_id": "...",  
  "data": {}  
}
```

3.31 Dependency Injection

FastAPI dependency injection shall be used for:

Database Sessions

Authentication

Authorization

Configuration

Telemetry

Repositories

Benefits:

Testability

Maintainability

Replaceability

3.32 Background Processing

Used for:

Audit generation

Telemetry processing

Notification processing

Heavy analysis jobs

Must remain observable and auditable.

3.33 Transaction Management

Transactions shall be managed at service layer.

Example:

Create Opportunity

↓

Create Evidence Link

↓

Create Audit Record

↓

Commit

Rollback on failure.

3.34 Backend Testing Structure

backend/tests/

- unit/
- integration/
- api/
- governance/
- performance/

Coverage required for:

Domains

Services

APIs

Governance

3.35 Performance Requirements

API Latency:
P95 < 300 ms

Search:
P95 < 2 s

Ranking:
P95 < 1 s

Recommendation:
P95 < 500 ms

Audit Generation:
P95 < 100 ms

3.36 Backend Governance Rules

BE-GOV-01

Business rules belong in domains.

BE-GOV-02

Repositories contain no business logic.

BE-GOV-03

Ranking occurs only in Ranking Domain.

BE-GOV-04

Recommendations depend on rankings.

BE-GOV-05

Governance controls cannot be bypassed.

BE-GOV-06

Auditability remains mandatory.

3.37 Architectural Constraints

AC-BE-01

API layer cannot contain business rules.

AC-BE-02

Repository layer cannot contain business rules.

AC-BE-03

Ranking logic is centralized.

AC-BE-04

Authority validation is centralized.

AC-BE-05

Evidence validation is centralized.

AC-BE-06

Governance controls remain enforceable.

3.38 Section Summary

The TODP Backend Architecture establishes FastAPI as the authoritative execution platform for all business, governance, ranking, recommendation, authority, and evidence workflows.

The architecture follows:

API Layer

↓

Application Services

↓

Domain Services

↓

Repositories

↓

Database

while preserving traceability, auditability, reproducibility, and governance integrity throughout the implementation lifecycle.

IAD Progress

- ☐ Section 1 — Repository Structure & Implementation Foundation
- ☐ Section 2 — Frontend Implementation Architecture (React)
- ☐ Section 3 — Backend Implementation Architecture (FastAPI)
- ☐ Section 4 — Database Schema Architecture & PostgreSQL Implementation

IAD V1 — Section 4

Database Schema Architecture & PostgreSQL Implementation

Document: Implementation Architecture Document (IAD V1)

Section: 4

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
 - ADD V1 (Frozen)
 - DDD V1 (Frozen)
 - SDD V1 (Frozen)
-

4.1 Purpose

This section defines the PostgreSQL database architecture, schema organization, table design principles, relationships, indexing strategy, and persistence implementation for TODP.

The database serves as the authoritative system of record for:

- Opportunities
 - Authorities
 - Evidence
 - Rankings
 - Recommendations
 - Users
 - Audit Records
 - Traceability Records
 - Governance Records
-

4.2 Database Principles

DB-01

PostgreSQL is the source of truth.

DB-02

Redis is never a source of truth.

DB-03

Audit data is immutable.

DB-04

Historical data must be preserved.

DB-05

Traceability must be maintained.

DB-06

Schema evolution must be versioned.

4.3 Technology Stack

Database:

PostgreSQL

Vector Search:

pgvector

ORM:

SQLAlchemy

Migration Tool:

Alembic

Driver:

psycopg

4.4 Database Structure

todp_db

- public
- audit
- governance
- observability

Purpose:

public
Business data

audit
Audit records

governance
Governance records

observability
Operational records

4.5 Core Entity Groups

Group 1

Users

Group 2

Opportunities

Group 3

Authorities

Group 4

Evidence

Group 5

Rankings

Group 6

Recommendations

Group 7

Audit & Governance

4.6 User Tables

users

Stores platform users.

Columns:

id

email

full_name

status

created_at

updated_at

user_preferences

Stores user preferences.

Columns:

id

user_id

key

value

created_at

4.7 Opportunity Tables

opportunities

Primary opportunity record.

Columns:

id

title

description

category

status

source_type

created_at

updated_at

4.8 Opportunity Metadata

opportunity_metadata

Stores structured attributes.

Columns:

id

opportunity_id

attribute_name

attribute_value

Purpose:

Flexible metadata expansion.

4.9 Authority Tables

authorities

Stores authority entities.

Columns:

id

name

authority_type

verification_status

confidence_score

created_at

updated_at

4.10 Authority Verification

authority_verifications

Stores verification history.

Columns:

id

authority_id

verification_result

verification_notes

verified_at

verified_by

Purpose:

Historical verification lineage.

4.11 Evidence Tables

evidence

Stores evidence records.

Columns:

id

title

description

source

status

created_at

updated_at

4.12 Evidence Versioning

evidence_versions

Stores historical evidence versions.

Columns:

id

evidence_id

version_number

change_summary

created_at

Purpose:

Reproducibility support.

4.13 Opportunity Authority Mapping

opportunity_authorities

Many-to-many relationship.

Columns:

opportunity_id

authority_id

created_at

Purpose:

Authority lineage.

4.14 Opportunity Evidence Mapping

opportunity_evidence

Many-to-many relationship.

Columns:

opportunity_id

evidence_id

created_at

Purpose:

Evidence lineage.

4.15 Ranking Tables

rankings

Stores ranking executions.

Columns:

id

query_id

generated_at

generated_by

4.16 Ranking Results

ranking_results

Stores ranked opportunities.

Columns:

id

ranking_id

opportunity_id

money_cost

time_cost

distance_cost

total_cost

rank_position

Purpose:

Ranking reproducibility.

4.17 Recommendation Tables

recommendations

Stores recommendation executions.

Columns:

id

user_id

ranking_id

generated_at

4.18 Recommendation Results

recommendation_results

Stores recommendation outputs.

Columns:

id

recommendation_id

opportunity_id

recommendation_reason

created_at

Purpose:

Recommendation traceability.

4.19 Opportunity Creation Tables

opportunity_concepts

Stores generated opportunity concepts.

Columns:

id

title

description

created_by

created_at

4.20 Similarity Analysis Tables

similarity_analysis

Stores similarity results.

Columns:

id

concept_id

similarity_score

reference_entity

created_at

4.21 Audit Tables

audit.audit_records

Stores immutable audit records.

Columns:

id

entity_type

entity_id

action

actor

timestamp

correlation_id

4.22 Traceability Tables

audit.trace_records

Stores lineage records.

Columns:

id

correlation_id

source_entity

target_entity

relationship_type

timestamp

4.23 Governance Tables

governance.decisions

Stores governance decisions.

Columns:

id

decision_name

decision_summary

approved_by

approved_at

4.24 Policy Tables

governance.policies

Stores governance policies.

Columns:

id

policy_name

policy_version

status

created_at

4.25 Vector Search Tables

opportunity_embeddings

Stores vectors.

Columns:

id

opportunity_id

embedding VECTOR

created_at

Purpose:

Semantic retrieval.

4.26 Search Query Tables

search_queries

Stores search activity.

Columns:

id

user_id

query_text

created_at

Purpose:

Search analytics and auditability.

4.27 Relationship Overview

User

↓

Search

↓

Ranking

↓

Recommendation

↓

Opportunity

↓

Authority

↓

Evidence

4.28 Primary Key Standard

All tables use:

UUID

Example:

id UUID PRIMARY KEY

Benefits:

Scalability

Distribution readiness

Security

4.29 Foreign Key Standard

Every relationship must enforce referential integrity.

Example:

opportunity_id

REFERENCES opportunities(id)

4.30 Indexing Strategy

Required Indexes:

Primary Keys

Foreign Keys

Search Fields

Audit Queries

Trace Queries

Ranking Queries

4.31 Search Indexes

Examples:

title

category

status

authority_type

verification_status

Purpose:

Fast retrieval.

4.32 Vector Indexes

Required for:

opportunity_embeddings

Index Type:

HNSW

Purpose:

Efficient semantic search.

4.33 Audit Indexes

Indexes:

timestamp

entity_type

correlation_id

actor

Purpose:

Fast investigations.

4.34 Soft Delete Strategy

Business entities use:

deleted_at

timestamp

instead of hard delete.

Benefits:

Auditability

Recovery

4.35 Historical Preservation

The following must preserve history:

Authorities

Evidence

Rankings

Recommendations

Governance Decisions

4.36 Migration Architecture

Alembic migration structure:

versions/

001_initial_schema.py

002_authority_schema.py

003_evidence_schema.py

004_ranking_schema.py

All schema changes must be versioned.

4.37 Database Governance Rules

DB-GOV-01

No direct production schema changes.

DB-GOV-02

All changes use migrations.

DB-GOV-03

Audit data is immutable.

DB-GOV-04

Historical records are preserved.

DB-GOV-05

Referential integrity is mandatory.

DB-GOV-06

Schema evolution is traceable.

4.38 Architectural Constraints

AC-DB-01

PostgreSQL remains source of truth.

AC-DB-02

Redis never becomes source of truth.

AC-DB-03

Audit records remain immutable.

AC-DB-04

Ranking history remains preserved.

AC-DB-05

Recommendation history remains preserved.

AC-DB-06

Lineage information remains recoverable.

4.39 Section Summary

The TODP Database Architecture establishes PostgreSQL and pgvector as the authoritative persistence platform.

The schema supports:

Users



Opportunities



Authorities



Evidence



Rankings



Recommendations



Governance



Auditability

while preserving integrity, traceability, auditability, reproducibility, and long-term platform governance requirements.

IAD Progress

- ☐ Section 1 — Repository Structure & Implementation Foundation
- ☐ Section 2 — Frontend Implementation Architecture (React)
- ☐ Section 3 — Backend Implementation Architecture (FastAPI)

□ Section 4 — Database Schema Architecture & PostgreSQL Implementation

□ Section 5 — Retrieval Implementation Architecture (BM25 + pgvector + Hybrid Search)

IAD V1 — Section 5

Retrieval Implementation Architecture (BM25 + pgvector + Hybrid Search)

Document: Implementation Architecture Document (IAD V1)

Section: 5

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)

- ADD V1 (Frozen)

- DDD V1 (Frozen)

- SDD V1 (Frozen)

5.1 Purpose

This section defines the implementation architecture for the TODP Retrieval System.

The retrieval system is responsible for:

- Opportunity retrieval
- Authority retrieval
- Evidence retrieval
- Semantic discovery
- Keyword discovery
- Hybrid search execution

Retrieval is responsible for finding candidates.

Retrieval does not perform ranking.

Retrieval does not perform recommendation.

5.2 Retrieval Philosophy

Discovery must maximize relevant opportunity recall.

Ranking determines priority.

Recommendation determines presentation.

Therefore:

Retrieval



Ranking



Recommendation

5.3 Retrieval Principles

RET-01

Retrieval and ranking remain separate.

RET-02

Keyword retrieval remains supported.

RET-03

Semantic retrieval remains supported.

RET-04

Hybrid retrieval is default.

RET-05

Retrieval remains explainable.

RET-06

Retrieval remains observable.

5.4 Retrieval Architecture

User Query

↓

Query Processing

↓

BM25 Retrieval

+

Vector Retrieval



Hybrid Fusion



Candidate Set



Ranking Engine

5.5 Retrieval Components

Component 1

Query Processor

Component 2

BM25 Engine

Component 3

Embedding Generator

Component 4

Vector Engine

Component 5

Hybrid Fusion Engine

Component 6

Retrieval Audit Layer

5.6 Backend Structure

backend/app/domains/retrieval/

- |— query_processor.py
- |— bm25_engine.py
- |— embedding_service.py
- |— vector_engine.py
- |— hybrid_engine.py
- |— retrieval_service.py
- |— retrieval_explainer.py

└─ retrieval_audit.py

5.7 Query Processing

Responsibilities:

- Query normalization
- Query cleaning
- Stop-word removal
- Query validation
- Query enrichment

Input:

"cheap online MBA scholarship"

Output:

normalized search query

5.8 Query Processing Pipeline

User Query

↓

Normalize

↓

Clean

↓

Validate

↓

Generate Embedding

↓

Execute Retrieval

5.9 BM25 Retrieval

Purpose:

Keyword-based retrieval.

Technology:

PostgreSQL Full Text Search

Implementation:

tsvector

tsquery

GIN indexes

5.10 BM25 Indexed Fields

Opportunities:

title

description

category

tags

Authority:

name

description

Evidence:

title

description

source

5.11 BM25 Example

User Query:

government scholarship

Generated Query:

government & scholarship

Result:

Keyword match candidates

5.12 BM25 Service

Class:

BM25Engine

Functions:

search()

score()

retrieve()

explain()

5.13 Embedding Service

Purpose:

Generate vector embeddings.

Input:

Text

Output:

Embedding Vector

5.14 Embedding Architecture

Query Text



Embedding Model



Vector



pgvector Search

5.15 Embedding Storage

Table:

opportunity_embeddings

Columns:

id

opportunity_id

embedding

created_at

5.16 Vector Search

Purpose:

Semantic similarity retrieval.

Technology:

pgvector

Index:

HNSW

Distance Metric:

Cosine Similarity

5.17 Vector Engine

Class:

VectorEngine

Functions:

generate_embedding()

search()

similarity_score()

retrieve()

5.18 Semantic Search Example

Opportunity:

"Financial Assistance Program"

Query:

"scholarship"

Keyword Match:

Weak

Semantic Match:

Strong

Vector retrieval identifies relationship.

5.19 Hybrid Retrieval

Purpose:

Combine BM25 and Vector Search.

Reason:

BM25 captures explicit intent.

Vector search captures semantic intent.

Hybrid captures both.

5.20 Hybrid Retrieval Architecture

BM25 Results

+

Vector Results

↓

Fusion Layer

↓

Combined Candidate Set

5.21 Hybrid Fusion Strategy

Initial V1 Strategy:

Weighted Score Fusion

Hybrid Score

=

(BM25 Score × Weight)

+

(Vector Score × Weight)

5.22 Initial Weighting

BM25 Weight

0.5

Vector Weight

0.5

Future tuning allowed through governance.

5.23 Candidate Generation

Retrieval Output:

Top N Candidates

Default:

Top 100

Candidates passed to ranking engine.

5.24 Candidate Rules

RET-CAND-01

Retrieval never decides final order.

RET-CAND-02

Retrieval never filters by ranking.

RET-CAND-03

Retrieval only produces candidate set.

5.25 Authority Retrieval

Purpose:

Find authority records related to opportunities.

Sources:

Authority Database

Authority Metadata

Authority Relationships

5.26 Evidence Retrieval

Purpose:

Find evidence supporting opportunities.

Sources:

Evidence Repository

Evidence Relationships

Evidence Metadata

5.27 Retrieval Service

Class:

RetrievalService

Responsibilities:

Execute BM25

Execute Vector Search

Execute Fusion

Generate Candidate Set

Generate Retrieval Metadata

5.28 Retrieval Metadata

Stored Data:

query

retrieval_time

bm25_results

vector_results

fusion_results

candidate_count

correlation_id

5.29 Retrieval Explanation

Every retrieval must support explanation.

Example:

Retrieved because:

Keyword Match:

"scholarship"

Semantic Match:

"financial assistance"

5.30 Retrieval Audit

Every retrieval generates:

Retrieval Event



Audit Record



Trace Record



Correlation ID

5.31 Search Query Storage

Table:

search_queries

Stored Fields:

query_text

user_id

timestamp

correlation_id

execution_time

5.32 Retrieval Caching

Technology:

Redis

Cache:

Query Results

Embedding Results

Frequently Used Searches

Rules:

Cache is not source of truth.

5.33 Retrieval Performance Targets

Query Processing:

P95 < 50 ms

BM25 Search:

P95 < 300 ms

Vector Search:

P95 < 500 ms

Hybrid Search:

P95 < 1 second

5.34 Observability

Metrics:

Search Count

Search Latency

BM25 Latency

Vector Latency

Fusion Latency

Cache Hit Rate

5.35 Error Handling

Failure Types:

Invalid Query

Embedding Failure

BM25 Failure

Vector Failure

Fusion Failure

Fallback:

BM25-only mode

if vector search unavailable.

5.36 Testing Requirements

Unit Tests

Query Processing

BM25 Engine

Vector Engine

Hybrid Engine

Integration Tests

Database

pgvector

Redis

Performance Tests

Search Throughput

Latency

Scalability

5.37 Governance Rules

RET-GOV-01

Retrieval remains independent from ranking.

RET-GOV-02

Retrieval remains auditable.

RET-GOV-03

Retrieval remains traceable.

RET-GOV-04

Retrieval explanations remain available.

RET-GOV-05

Hybrid weights remain governed.

5.38 Architectural Constraints

AC-RET-01

Retrieval does not rank.

AC-RET-02

Retrieval does not recommend.

AC-RET-03

Retrieval does not override governance.

AC-RET-04

Retrieval remains reproducible.

AC-RET-05

Retrieval history remains preserved.

AC-RET-06

Hybrid search remains explainable.

5.39 Section Summary

The TODP Retrieval Architecture implements a hybrid discovery model combining:

BM25 Keyword Search

+

Vector Semantic Search



Hybrid Fusion



Candidate Generation



Ranking Engine

This architecture maximizes discovery quality while preserving transparency, traceability, auditability, and reproducibility.

IAD Progress

🔗 Section 1 — Repository Structure & Implementation Foundation

🔗 Section 2 — Frontend Implementation Architecture (React)

🔗 Section 3 — Backend Implementation Architecture (FastAPI)

🔗 Section 4 — Database Schema Architecture & PostgreSQL Implementation

🔗 Section 5 — Retrieval Implementation Architecture (BM25 + pgvector + Hybrid Search)

▣ Section 6 — Ranking Engine Implementation Architecture

IAD V1 — Section 6

Ranking Engine Implementation Architecture

Document: Implementation Architecture Document (IAD V1)

Section: 6

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)

6.1 Purpose

This section defines the implementation architecture for the TODP Ranking Engine.

The Ranking Engine is responsible for:

- Candidate evaluation
- Cost calculation
- Opportunity ordering
- Ranking explanation
- Ranking reproducibility
- Ranking traceability

The Ranking Engine transforms retrieval candidates into ordered opportunities.

6.2 Ranking Philosophy

TODP does not rank by:

- Popularity
- Engagement
- Sponsorship
- Advertising
- Hidden weights

TODP ranks using:

Total Cost

=

Money Cost

+

Time Cost

+

Distance Cost

Lowest Total Cost ranks highest.

6.3 Ranking Principles

RANK-01

Ranking occurs after retrieval.

RANK-02

Ranking occurs before recommendation.

RANK-03

Ranking must be explainable.

RANK-04

Ranking must be reproducible.

RANK-05

Ranking must be auditable.

RANK-06

Ranking logic must remain transparent.

6.4 Ranking Workflow

Retrieval Candidates

↓

Cost Calculation

↓

Score Generation



Ranking Sort



Explanation Generation



Recommendation Engine

6.5 Ranking Module Structure

backend/app/domains/ranking/

|— ranking_engine.py

|— cost_calculator.py

|— money_cost_service.py

|— time_cost_service.py

|— distance_cost_service.py

|— ranking_explainer.py

|— ranking_audit.py

└─ ranking_service.py

6.6 Ranking Components

Component 1

Candidate Processor

Component 2

Money Cost Calculator

Component 3

Time Cost Calculator

Component 4

Distance Cost Calculator

Component 5

Ranking Engine

Component 6

Explanation Engine

Component 7

Audit Layer

6.7 Candidate Input

Input Source:

Retrieval Service

Input:

Candidate Opportunity Set

Example:

Candidate A

Candidate B

Candidate C

Candidate D

6.8 Candidate Validation

Before ranking:

Validate Opportunity

Validate Authority Links

Validate Evidence Links

Validate Required Metadata

Remove Invalid Candidates

6.9 Money Cost

Definition:

Direct financial expenditure required to access opportunity.

Examples:

Application Fees

Registration Fees

Travel Expenses

Tuition Costs

Material Costs

6.10 Money Cost Structure

Class:

MoneyCostService

Functions:

calculate()

validate()

normalize()

6.11 Time Cost

Definition:

Amount of time required.

Examples:

Application Time

Travel Time

Processing Time

Participation Time

6.12 Time Cost Structure

Class:

TimeCostService

Functions:

calculate()

normalize()

validate()

6.13 Distance Cost

Definition:

Physical distance burden.

Examples:

Online Opportunity

Distance = 0

Local Opportunity

Distance = Local Distance

Physical Opportunity

Distance = Actual Travel Distance

6.14 Distance Cost Structure

Class:

DistanceCostService

Functions:

calculate()

normalize()

validate()

6.15 Total Cost Formula

Total Cost

=

Money Cost

+

Time Cost

+

Distance Cost

6.16 Cost Calculation Service

Class:

CostCalculator

Responsibilities:

Calculate Money Cost

Calculate Time Cost

Calculate Distance Cost

Calculate Total Cost

Return Cost Breakdown

6.17 Example Calculation

Opportunity A

Money Cost = 20

Time Cost = 10

Distance Cost = 5

Total Cost = 35

Opportunity B

Money Cost = 15

Time Cost = 15

Distance Cost = 10

Total Cost = 40

Result

Opportunity A ranks above Opportunity B

6.18 Ranking Engine

Class:

RankingEngine

Functions:

rank()

sort()

resolve_ties()

generate_results()

6.19 Ranking Algorithm

Step 1

Receive Candidates



Step 2

Calculate Costs



Step 3

Calculate Total Cost



Step 4

Sort Ascending



Step 5

Generate Ranking Positions



Step 6

Generate Explanations

6.20 Sorting Rule

Ascending Total Cost

Example:

25

35

50

75

100

Lowest value receives Rank 1

6.21 Tie Resolution

When Total Cost is identical:

Rule 1

Lower Money Cost wins

If equal:

Rule 2

Lower Time Cost wins

If equal:

Rule 3

Lower Distance Cost wins

If equal:

Rule 4

Stable deterministic ordering

6.22 Ranking Result Model

Ranking Result:

ranking_id

opportunity_id

money_cost

time_cost

distance_cost

total_cost

rank_position

timestamp

6.23 Ranking Persistence

Tables:

rankings

ranking_results

Purpose:

Historical reconstruction

Auditability

Reproducibility

6.24 Ranking Explanation Engine

Class:

RankingExplainer

Responsibilities:

Explain ranking

Explain cost calculation

Explain rank position

Generate transparency data

6.25 Example Explanation

Rank: 1

Money Cost: 20

Time Cost: 10

Distance Cost: 5

Total Cost: 35

Reason:

Lowest Total Cost among candidates.

6.26 Ranking Traceability

Each ranking stores:

Input Query

Candidate Set

Cost Values

Formula Version

Ranking Version

Correlation ID

Timestamp

6.27 Ranking Reproducibility

Required Inputs:

Opportunity Data

Authority Data

Evidence Data

Formula Version

Ranking Version

Timestamp

Same Inputs

↓

Same Output

6.28 Ranking Audit

Every ranking execution generates:

Ranking Event

↓

Audit Record



Trace Record



Lineage Record

6.29 Ranking Versioning

Table:

ranking_versions

Columns:

version

description

effective_date

status

Purpose:

Historical reproducibility.

6.30 Future Formula Evolution

Future versions may enhance:

Money Cost Estimation

Time Cost Estimation

Distance Cost Estimation

Constraint:

Core Formula remains transparent.

6.31 Ranking API

Example Endpoint

POST

/api/v1/ranking/execute

Input:

query_id

candidate_set

Output:

ranked opportunities

6.32 Performance Requirements

Cost Calculation:

P95 < 100 ms

Ranking Execution:

P95 < 500 ms

Explanation Generation:

P95 < 100 ms

Ranking Storage:

P95 < 100 ms

6.33 Observability

Metrics:

Ranking Count

Ranking Duration

Average Candidate Count

Ranking Errors

Tie Resolution Count

6.34 Testing Requirements

Unit Tests

Cost Calculators

Ranking Engine

Tie Resolution

Explanation Engine

Integration Tests

Database

Retrieval Integration

Recommendation Integration

Regression Tests

Ranking Stability

Historical Reproducibility

6.35 Governance Rules

RANK-GOV-01

Ranking logic remains transparent.

RANK-GOV-02

Ranking logic remains reproducible.

RANK-GOV-03

Ranking logic remains auditable.

RANK-GOV-04

Ranking history remains preserved.

RANK-GOV-05

Ranking explanations remain available.

6.36 Architectural Constraints

AC-RANK-01

No sponsored ranking.

AC-RANK-02

No paid placement.

AC-RANK-03

No hidden weighting.

AC-RANK-04

Ranking occurs before recommendation.

AC-RANK-05

AI cannot override rankings.

AC-RANK-06

Ranking formula remains visible.

6.37 Section Summary

The TODP Ranking Engine transforms retrieval candidates into ordered opportunities using a transparent Total Cost framework.

Retrieval Candidates

↓

Money Cost

+

Time Cost

+

Distance Cost

↓

Total Cost



Ranking



Explanation



Recommendation

The architecture ensures transparency, auditability, traceability, reproducibility, and strict alignment with TODP's foundational principles.

IAD Progress

📄 Section 1 — Repository Structure & Implementation Foundation

📄 Section 2 — Frontend Implementation Architecture (React)

📄 Section 3 — Backend Implementation Architecture (FastAPI)

📄 Section 4 — Database Schema Architecture & PostgreSQL Implementation

📄 Section 5 — Retrieval Implementation Architecture (BM25 + pgvector + Hybrid Search)

▣ Section 6 — Ranking Engine Implementation Architecture

▣ Section 7 — Recommendation Engine Implementation Architecture

IAD V1 — Section 7

Recommendation Engine Implementation Architecture

Document: Implementation Architecture Document (IAD V1)

Section: 7

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)

7.1 Purpose

This section defines the implementation architecture for the TODP Recommendation Engine.

The Recommendation Engine is responsible for:

- Recommendation generation
- Recommendation explanation
- Evidence attachment
- Authority attachment
- Recommendation traceability
- Recommendation reproducibility

The Recommendation Engine does not create rankings.

The Recommendation Engine consumes rankings.

7.2 Recommendation Philosophy

TODP recommendations are not opinions.

TODP recommendations are structured presentations of ranked opportunities.

Principle:

Evidence



Authority



Ranking



Recommendation

Recommendations cannot bypass this sequence.

7.3 Recommendation Principles

REC-01

Recommendations depend on rankings.

REC-02

Recommendations must include supporting evidence.

REC-03

Recommendations must include supporting authorities.

REC-04

Recommendations must remain explainable.

REC-05

Recommendations must remain auditable.

REC-06

Recommendations must remain reproducible.

7.4 Recommendation Workflow

User Request



Retrieval



Ranking



Recommendation Assembly



Explanation Generation

↓

Response

7.5 Recommendation Module Structure

backend/app/domains/recommendation/

- |— recommendation_engine.py
- |— recommendation_service.py
- |— recommendation_builder.py
- |— recommendation_explainer.py
- |— recommendation_validator.py
- |— recommendation_audit.py
- └— recommendation_repository.py

7.6 Recommendation Components

Component 1

Recommendation Builder

Component 2

Authority Collector

Component 3

Evidence Collector

Component 4

Explanation Generator

Component 5

Recommendation Validator

Component 6

Audit Layer

7.7 Recommendation Input

Input Sources:

- Ranking Results
- Authority Records
- Evidence Records
- User Context

Input Example:

ranking_id

user_id

candidate_set

authority_links

evidence_links

7.8 Recommendation Builder

Class:

RecommendationBuilder

Responsibilities:

- Assemble recommendation
- Attach evidence
- Attach authority
- Generate explanation structure

Output:

Recommendation Object

7.9 Recommendation Selection

Input:

Ranked Opportunities

Example:

Rank 1

Rank 2

Rank 3

Rank 4

Rank 5

Output:

Recommended Opportunities

Default:

Top 10

Configurable through governance.

7.10 Recommendation Validator

Class:

RecommendationValidator

Responsibilities:

- Verify ranking exists
- Verify evidence exists
- Verify authority exists
- Verify traceability exists

Invalid recommendations are rejected.

7.11 Authority Attachment

Every recommendation must reference:

Authority Name

Authority Type

Verification Status

Authority Identifier

Authority Source

7.12 Evidence Attachment

Every recommendation must reference:

Evidence Identifier

Evidence Source

Evidence Type

Evidence Status

Evidence References

7.13 Recommendation Object

Recommendation:

recommendation_id

opportunity_id

rank_position

authority_ids

evidence_ids

recommendation_reason

generated_at

7.14 Recommendation Explanation Engine

Class:

RecommendationExplainer

Responsibilities:

- Generate recommendation reasons
- Generate ranking explanation

- Generate authority explanation
- Generate evidence explanation

7.15 Explanation Structure

Example:

Recommended because:

Rank Position: 1

Money Cost: 20

Time Cost: 10

Distance Cost: 5

Total Cost: 35

Authority:

Verified Authority

Evidence:

Supporting Evidence Available

7.16 User Context Integration

Recommendation Engine may consider:

Preferred Location

Preferred Category

Preferred Format

Eligibility Constraints

User context influences presentation.

User context does not override rankings.

7.17 Recommendation Persistence

Tables:

recommendations

recommendation_results

Purpose:

- Historical analysis
- Traceability
- Auditability
- Reproducibility

7.18 Recommendation Lifecycle

Stage 1

Recommendation Request

Stage 2

Ranking Retrieval

Stage 3

Authority Retrieval

Stage 4

Evidence Retrieval

Stage 5

Recommendation Assembly

Stage 6

Explanation Generation

Stage 7

Persistence

7.19 Recommendation Traceability

Every recommendation stores:

recommendation_id

ranking_id

authority_ids

evidence_ids

formula_version

correlation_id

generated_at

7.20 Recommendation Reproducibility

Required Inputs:

Ranking Results

Authority Data

Evidence Data

Recommendation Version

Timestamp

Same Inputs

↓

Same Recommendation Output

7.21 Recommendation Audit

Every recommendation generates:

Recommendation Event



Audit Record



Trace Record



Lineage Record



Correlation ID

7.22 Recommendation API

POST

/api/v1/recommendations/generate

Input:

user_id

ranking_id

Output:

recommendation_set

GET

/api/v1/recommendations/{id}

Returns:

Recommendation Details

Authorities

Evidence

Explanation

7.23 Recommendation Versioning

Table:

recommendation_versions

Columns:

version

description

effective_date

status

Purpose:

Historical reproducibility.

7.24 Recommendation History

All recommendation executions remain stored.

Benefits:

Auditability

Traceability

Governance Reviews

Historical Analysis

Reproducibility

7.25 Performance Requirements

Recommendation Assembly:

P95 < 300 ms

Authority Retrieval:

P95 < 200 ms

Evidence Retrieval:

P95 < 200 ms

Explanation Generation:

P95 < 100 ms

Total Recommendation Time:

P95 < 500 ms

7.26 Observability

Metrics:

Recommendation Count

Recommendation Latency

Authority Attachment Count

Evidence Attachment Count

Recommendation Errors

Explanation Generation Time

7.27 Error Handling

Failure Types:

Missing Ranking

Missing Authority

Missing Evidence

Recommendation Failure

Persistence Failure

Explanation Failure

Failures must be logged and audited.

7.28 Testing Requirements

Unit Tests:

Recommendation Builder

Recommendation Validator

Explanation Engine

Authority Attachment

Evidence Attachment

Integration Tests:

Ranking Integration

Authority Integration

Evidence Integration

Database Integration

Regression Tests:

Recommendation Stability

Historical Reproducibility

7.29 Governance Rules

REC-GOV-01

Recommendations require rankings.

REC-GOV-02

Recommendations require evidence.

REC-GOV-03

Recommendations require authorities.

REC-GOV-04

Recommendations remain auditable.

REC-GOV-05

Recommendations remain traceable.

REC-GOV-06

Recommendations remain reproducible.

7.30 Architectural Constraints

AC-REC-01

Recommendations cannot bypass rankings.

AC-REC-02

Recommendations cannot bypass evidence.

AC-REC-03

Recommendations cannot bypass authorities.

AC-REC-04

AI cannot override rankings.

AC-REC-05

Recommendation explanations remain available.

AC-REC-06

Recommendation history remains preserved.

7.31 Section Summary

The TODP Recommendation Engine transforms ranked opportunities into explainable recommendations by attaching verified authorities and supporting evidence.

Ranking Results



Authority Attachment



Evidence Attachment



Explanation Generation



Recommendation Output

The architecture ensures transparency, traceability, auditability, reproducibility, and strict adherence to TODP's core principles.

IAD Progress

📄 Section 1 — Repository Structure & Implementation Foundation

📄 Section 2 — Frontend Implementation Architecture (React)

📄 Section 3 — Backend Implementation Architecture (FastAPI)

📄 Section 4 — Database Schema Architecture & PostgreSQL Implementation

📄 Section 5 — Retrieval Implementation Architecture (BM25 + pgvector + Hybrid Search)

▢ Section 6 — Ranking Engine Implementation Architecture

▢ Section 7 — Recommendation Engine Implementation Architecture

▢ Section 8 — Authority Verification Implementation Architecture

IAD V1 — Section 8

Authority Verification Implementation Architecture

Document: Implementation Architecture Document (IAD V1)

Section: 8

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)

- ADD V1 (Frozen)

- DDD V1 (Frozen)

- SDD V1 (Frozen)

8.1 Purpose

This section defines the implementation architecture for Authority Verification within TODP.

Authority Verification is responsible for:

- Authority registration
- Authority validation
- Authority lifecycle management
- Authority traceability
- Authority auditability
- Authority confidence management

Authority Verification supports the principle:

Authority Before Recommendation

No recommendation may reference an unverified authority.

8.2 Authority Philosophy

Authority is not determined by:

- Popularity
- Social engagement
- AI opinion
- User votes

Authority is determined by:

- Source legitimacy

- Verification evidence
- Governance validation
- Traceable records

8.3 Authority Principles

AUTH-01

Authorities must be verifiable.

AUTH-02

Authorities must be traceable.

AUTH-03

Authorities must be auditable.

AUTH-04

Authority status must be reproducible.

AUTH-05

Authority history must be preserved.

AUTH-06

AI cannot act as authority.

8.4 Authority Workflow

Authority Registration



Authority Validation



Verification Decision



Authority Approval



Authority Usage

↓

Periodic Review

8.5 Authority Module Structure

backend/app/domains/authority/

|— authority_service.py

|— authority_validator.py

|— authority_verifier.py

|— authority_repository.py

|— authority_lifecycle.py

|— authority_explainer.py

|— authority_audit.py

|— authority_rules.py

8.6 Authority Components

Component 1

Authority Registry

Component 2

Verification Engine

Component 3

Confidence Engine

Component 4

Lifecycle Manager

Component 5

Explanation Engine

Component 6

Audit Layer

8.7 Authority Entity

Authority Object:

authority_id

authority_name

authority_type

verification_status

confidence_score

created_at

updated_at

8.8 Authority Types

Examples:

Institution

Government Body

University

Organization

Research Body

Certification Body

Public Agency

Industry Authority

8.9 Authority Registration

Input:

Authority Name

Authority Type

Description

Reference Sources

Supporting Evidence

Output:

Authority Record

Verification Pending

8.10 Verification Status Model

Possible States:

PENDING

UNDER_REVIEW

VERIFIED

REJECTED

SUSPENDED

ARCHIVED

8.11 Verification Engine

Class:

AuthorityVerifier

Responsibilities:

- Execute verification checks
- Evaluate legitimacy
- Generate verification outcome
- Store verification history

8.12 Verification Inputs

Authority Name

Authority Type

Supporting Sources

Supporting Evidence

Verification Metadata

Governance Rules

8.13 Verification Rules

Example Rules:

Authority exists.

Authority is identifiable.

Authority has valid references.

Authority has supporting evidence.

Authority satisfies governance requirements.

8.14 Verification Decision

Possible Outcomes:

VERIFIED

REJECTED

REVIEW_REQUIRED

SUSPENDED

8.15 Confidence Score

Purpose:

Measure verification confidence.

Range:

0 - 100

Example:

95

Highly Verified

75

Moderately Verified

40

Requires Review

8.16 Confidence Factors

Verification Evidence

Source Quality

Authority Stability

Verification History

Governance Reviews

8.17 Authority Lifecycle

Stage 1

Registered

Stage 2

Validated

Stage 3

Verified

Stage 4

Operational

Stage 5

Review

Stage 6

Archived

8.18 Authority Review Process

Periodic reviews validate:

Authority still exists

Authority remains valid

Evidence remains current

Verification remains accurate

8.19 Authority Relationships

Tables:

authorities

authority_verifications

opportunity_authorities

Purpose:

Authority lineage.

Authority traceability.

8.20 Authority Traceability

Every authority stores:

authority_id

verification_id

supporting_evidence

review_history

correlation_id

timestamps

8.21 Authority Explanation Engine

Class:

AuthorityExplainer

Responsibilities:

Explain:

Why authority is trusted

Why authority is verified

Which evidence supports authority

8.22 Example Explanation

Authority:

XYZ University

Status:

Verified

Reason:

Registered institution

Verification Evidence Available

Governance Validation Passed

Confidence Score: 92

8.23 Authority Usage Rules

Verified Authorities:

Allowed

Pending Authorities:

Restricted

Rejected Authorities:

Blocked

Suspended Authorities:

Blocked

8.24 Recommendation Integration

Recommendation Engine requires:

Verified Authority



Authority Reference



Recommendation Generation

No authority



No recommendation

8.25 Ranking Integration

Ranking Engine may reference:

Authority Metadata

Authority Relationships

Authority Availability

Ranking never modifies authority status.

8.26 Audit Architecture

Every authority action generates:

Authority Event



Audit Record



Trace Record



Lineage Record



Correlation ID

8.27 Authority History

Historical records preserved:

Verification Events

Status Changes

Review Events

Governance Decisions

Confidence Changes

8.28 Authority API

POST

/api/v1/authorities

Create authority

GET

/api/v1/authorities/{id}

Retrieve authority

POST

/api/v1/authorities/{id}/verify

Execute verification

GET

/api/v1/authorities/{id}/history

Retrieve authority history

8.29 Database Tables

authorities

authority_verifications

opportunity_authorities

audit_records

trace_records

8.30 Performance Requirements

Authority Retrieval:

P95 < 100 ms

Authority Verification:

P95 < 2 s

Authority History Retrieval:

P95 < 300 ms

Authority Explanation:

P95 < 100 ms

8.31 Observability

Metrics:

Authority Count

Verification Count

Verification Success Rate

Verification Failures

Review Count

Confidence Distribution

8.32 Error Handling

Failure Types:

Missing Authority

Verification Failure

Review Failure

Persistence Failure

Governance Failure

All failures must be logged and audited.

8.33 Testing Requirements

Unit Tests:

Authority Validator

Authority Verifier

Confidence Engine

Authority Lifecycle

Explanation Engine

Integration Tests:

Database Integration

Evidence Integration

Recommendation Integration

Governance Integration

Regression Tests:

Authority Reproducibility

Verification Consistency

8.34 Governance Rules

AUTH-GOV-01

Authorities must be verified.

AUTH-GOV-02

Authority history must be preserved.

AUTH-GOV-03

Authority decisions must be auditable.

AUTH-GOV-04

Authority decisions must be traceable.

AUTH-GOV-05

Authority confidence must be measurable.

AUTH-GOV-06

Authority reviews must be documented.

8.35 Architectural Constraints

AC-AUTH-01

AI cannot become authority.

AC-AUTH-02

Recommendations require verified authorities.

AC-AUTH-03

Authority decisions remain reproducible.

AC-AUTH-04

Authority history remains available.

AC-AUTH-05

Verification logic remains auditable.

AC-AUTH-06

Governance controls cannot be bypassed.

8.36 Section Summary

The TODP Authority Verification Architecture establishes a governed framework for validating and managing trusted authorities.

Authority Registration

↓

Verification



Confidence Assessment



Approval



Recommendation Usage



Periodic Review

The architecture ensures that all authorities remain verifiable, traceable, auditable, reproducible, and aligned with TODP's foundational principle:

Authority Before Recommendation.

IAD Progress

🔗 Section 1 — Repository Structure & Implementation Foundation

🔗 Section 2 — Frontend Implementation Architecture (React)

🔗 Section 3 — Backend Implementation Architecture (FastAPI)

- 🔗 Section 4 — Database Schema Architecture & PostgreSQL Implementation
- 🔗 Section 5 — Retrieval Implementation Architecture (BM25 + pgvector + Hybrid Search)
- 🔗 Section 6 — Ranking Engine Implementation Architecture
- 🔗 Section 7 — Recommendation Engine Implementation Architecture
- 🔗 Section 8 — Authority Verification Implementation Architecture
- 🔗 Section 9 — Evidence Validation Implementation Architecture

IAD V1 — Section 9

Evidence Validation Implementation Architecture

Document: Implementation Architecture Document (IAD V1)

Section: 9

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)

9.1 Purpose

This section defines the implementation architecture for Evidence Validation within TODP.

Evidence Validation is responsible for:

- Evidence registration
- Evidence validation
- Evidence lineage management
- Evidence version management
- Evidence traceability
- Evidence auditability

Evidence Validation supports the principle:

Evidence First

Evidence is the foundation upon which authorities, rankings, and recommendations are built.

9.2 Evidence Philosophy

Evidence is not:

- Opinion
- Assumption
- Popularity
- AI-generated belief

Evidence is:

- Observable
- Traceable
- Verifiable
- Reproducible
- Governed

Every important platform outcome must be supported by evidence.

9.3 Evidence Principles

EVD-01

Evidence must be identifiable.

EVD-02

Evidence must be traceable.

EVD-03

Evidence must be auditable.

EVD-04

Evidence must be versioned.

EVD-05

Evidence must be reproducible.

EVD-06

Evidence history must be preserved.

9.4 Evidence Workflow

Evidence Registration



Evidence Validation



Evidence Approval



Evidence Usage

↓

Evidence Review

↓

Evidence Archival

9.5 Evidence Module Structure

backend/app/domains/evidence/

- |— evidence_service.py
- |— evidence_validator.py
- |— evidence_repository.py
- |— evidence_lifecycle.py
- |— evidence_versioning.py
- |— evidence_lineage.py
- |— evidence_explainer.py
- |— evidence_audit.py
- |— evidence_rules.py

9.6 Evidence Components

Component 1

Evidence Registry

Component 2

Validation Engine

Component 3

Version Manager

Component 4

Lineage Manager

Component 5

Explanation Engine

Component 6

Audit Layer

9.7 Evidence Entity

Evidence Object:

evidence_id

title

description

evidence_type

source

status

version

created_at

updated_at

9.8 Evidence Types

Examples:

Official Document

Government Notification

University Circular

Research Publication

Policy Document

Regulation

Program Announcement

Verification Record

9.9 Evidence Registration

Input:

Title

Description

Source

Evidence Type

Reference Metadata

Supporting Documents

Output:

Evidence Record

Status = PENDING

9.10 Evidence Status Model

Possible States:

PENDING

UNDER_REVIEW

VALIDATED

REJECTED

EXPIRED

ARCHIVED

9.11 Validation Engine

Class:

EvidenceValidator

Responsibilities:

- Validate evidence structure
- Validate source information
- Validate completeness
- Generate validation outcome

9.12 Validation Inputs

Evidence Metadata

Source Information

Supporting References

Evidence Type

Governance Rules

9.13 Validation Rules

Evidence must have title.

Evidence must have source.

Evidence must be identifiable.

Evidence must be traceable.

Evidence must satisfy governance requirements.

Evidence must support verification.

9.14 Validation Outcomes

VALIDATED

REJECTED

REVIEW_REQUIRED

EXPIRED

9.15 Evidence Versioning

Purpose:

Preserve historical evidence state.

Table:

evidence_versions

Fields:

evidence_id

version_number

change_summary

created_at

9.16 Version Lifecycle

Version 1

Initial Registration



Version 2

Metadata Update



Version 3

Evidence Revision



Version N

Historical Preservation

9.17 Evidence Lineage

Purpose:

Track evidence relationships.

Example:

Evidence

↓

Authority Verification

↓

Opportunity

↓

Ranking



Recommendation

9.18 Lineage Manager

Class:

EvidenceLineageManager

Responsibilities:

Create lineage records

Track relationships

Generate lineage graph

Support traceability

9.19 Evidence Relationships

Tables:

evidence

evidence_versions

opportunity_evidence

trace_records

audit_records

9.20 Evidence Traceability

Every evidence record stores:

evidence_id

source

version

validation_status

lineage_id

correlation_id

timestamps

9.21 Evidence Explanation Engine

Class:

EvidenceExplainer

Responsibilities:

Explain:

Why evidence is valid

How evidence is used

What decisions depend on evidence

9.22 Example Explanation

Evidence:

Government Scholarship Circular

Status:

Validated

Source:

Official Government Portal

Version:

3

Used By:

Scholarship Opportunity Ranking

Authority Verification

Recommendation Generation

9.23 Evidence Usage Rules

Validated Evidence:

Allowed

Pending Evidence:

Restricted

Rejected Evidence:

Blocked

Expired Evidence:

Blocked

Archived Evidence:

Historical Use Only

9.24 Authority Integration

Authority Verification may consume:

Validated Evidence



Authority Decision

No evidence



No authority verification

9.25 Ranking Integration

Ranking Engine may reference:

Evidence Relationships

Evidence Availability

Evidence Status

Ranking Engine does not modify evidence.

9.26 Recommendation Integration

Recommendations require:

Validated Evidence



Evidence References



Recommendation Output

No evidence



Recommendation blocked

9.27 Audit Architecture

Every evidence action generates:

Evidence Event



Audit Record



Trace Record



Lineage Record



Correlation ID

9.28 Evidence Review Process

Periodic review validates:

Source still exists

Evidence remains relevant

Evidence remains valid

Evidence status remains accurate

9.29 Evidence History

Historical records preserved:

Version Changes

Validation Events

Status Changes

Review Events

Governance Decisions

9.30 Evidence API

POST

/api/v1/evidence

Create evidence

GET

/api/v1/evidence/{id}

Retrieve evidence

POST

/api/v1/evidence/{id}/validate

Execute validation

GET

/api/v1/evidence/{id}/history

Retrieve history

GET

/api/v1/evidence/{id}/lineage

Retrieve lineage

9.31 Database Tables

evidence

evidence_versions

opportunity_evidence

audit_records

trace_records

9.32 Performance Requirements

Evidence Retrieval:

P95 < 100 ms

Evidence Validation:

P95 < 2 s

Evidence History Retrieval:

P95 < 300 ms

Evidence Lineage Retrieval:

P95 < 500 ms

Evidence Explanation:

P95 < 100 ms

9.33 Observability

Metrics:

Evidence Count

Validation Count

Validation Success Rate

Validation Failures

Review Count

Version Count

Lineage Count

9.34 Error Handling

Failure Types:

Missing Evidence

Validation Failure

Version Failure

Lineage Failure

Persistence Failure

Governance Failure

All failures must be logged and audited.

9.35 Testing Requirements

Unit Tests:

Evidence Validator

Version Manager

Lineage Manager

Evidence Lifecycle

Explanation Engine

Integration Tests:

Database Integration

Authority Integration

Recommendation Integration

Governance Integration

Regression Tests:

Evidence Reproducibility

Validation Consistency

Lineage Consistency

9.36 Governance Rules

EVD-GOV-01

Evidence must be validated.

EVD-GOV-02

Evidence history must be preserved.

EVD-GOV-03

Evidence lineage must be preserved.

EVD-GOV-04

Evidence decisions must be auditable.

EVD-GOV-05

Evidence decisions must be traceable.

EVD-GOV-06

Evidence reviews must be documented.

9.37 Architectural Constraints

AC-EVD-01

Evidence First remains mandatory.

AC-EVD-02

Authority verification requires evidence.

AC-EVD-03

Recommendations require evidence.

AC-EVD-04

Evidence history remains available.

AC-EVD-05

Evidence validation remains auditable.

AC-EVD-06

Governance controls cannot be bypassed.

9.38 Section Summary

The TODP Evidence Validation Architecture establishes a governed framework for validating, preserving, and tracing evidence throughout the platform.

Evidence Registration



Validation



Versioning



Lineage Tracking



Authority Verification



Ranking



Recommendation

The architecture ensures that evidence remains verifiable, traceable, auditable, reproducible, and aligned with TODP's foundational principle:

Evidence First.

IAD Progress

- 🔗 Section 2 — Frontend Implementation Architecture (React)
- 🔗 Section 3 — Backend Implementation Architecture (FastAPI)
- 🔗 Section 4 — Database Schema Architecture & PostgreSQL Implementation
- 🔗 Section 5 — Retrieval Implementation Architecture (BM25 + pgvector + Hybrid Search)
- 🔗 Section 6 — Ranking Engine Implementation Architecture
- 🔗 Section 7 — Recommendation Engine Implementation Architecture
- 🔗 Section 8 — Authority Verification Implementation Architecture
- 🔗 Section 9 — Evidence Validation Implementation Architecture

- 🔗 Section 10 — Auditability, Traceability & Reproducibility Implementation Architecture

IAD V1 — Section 10

Auditability, Traceability & Reproducibility Implementation Architecture

Document: Implementation Architecture Document (IAD V1)

Section: 10

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)

10.1 Purpose

This section defines the implementation architecture for Auditability, Traceability, and Reproducibility (ATR) within TODP.

ATR is responsible for:

- Audit record generation
- Decision trace generation
- Lineage generation
- Historical reconstruction
- Reproducibility validation
- Governance accountability

ATR is a platform-wide capability.

Every critical workflow must participate.

10.2 ATR Philosophy

Users must be able to answer:

Who made a decision?

What happened?

Why did it happen?

When did it happen?

What evidence was used?

What authority was used?

Can the result be reproduced?

If these questions cannot be answered, trust cannot be established.

10.3 Core Principles

ATR-01

Everything important is auditable.

ATR-02

Everything important is traceable.

ATR-03

Important outcomes are reproducible.

ATR-04

Lineage must be preserved.

ATR-05

Historical state must remain available.

ATR-06

Audit records are immutable.

10.4 ATR Architecture

User Action



Business Event



Audit Record

↓

Trace Record

↓

Lineage Record

↓

Historical Storage

↓

Reproducibility Support

10.5 ATR Module Structure

backend/app/governance/

├─ audit/

| └─ audit_service.py

| └─ audit_repository.py

| └─ audit_models.py

|


```
└─ traceability/
|   └─ trace_service.py
|   └─ lineage_service.py
|   └─ trace_models.py
|
└─ reproducibility/
|   └─ replay_service.py
|   └─ snapshot_service.py
|   └─ reproducibility_validator.py
```

10.6 Audit Architecture

Purpose:

Record important platform actions.

Examples:

Opportunity Creation

Authority Verification

Evidence Validation

Ranking Execution

Recommendation Generation

Governance Decisions

10.7 Audit Record Structure

audit_record

Fields:

audit_id

entity_type

entity_id

action

actor

timestamp

correlation_id

metadata

10.8 Audit Event Types

Examples:

CREATE

UPDATE

DELETE

VALIDATE

VERIFY

RANK

RECOMMEND

APPROVE

REJECT

LOGIN

LOGOUT

10.9 Audit Service

Class:

AuditService

Responsibilities:

Create audit records

Store audit records

Retrieve audit records

Search audit records

10.10 Audit Repository

Responsibilities:

Persistence

Filtering

Search

Pagination

Historical retrieval

10.11 Immutable Audit Design

Audit records:

Cannot be modified.

Cannot be deleted.

Can only be appended.

Purpose:

Preserve accountability.

10.12 Correlation ID Framework

Every workflow receives:

correlation_id

Example:

REQ-2026-000123

Purpose:

Connect all events within a workflow.

10.13 Correlation Flow Example

Search Request

↓

Retrieval Event

↓

Ranking Event



Recommendation Event



Audit Events



Shared Correlation ID

10.14 Traceability Architecture

Purpose:

Track relationships between entities.

Example:

Evidence



Authority



Opportunity



Ranking



Recommendation

10.15 Trace Record Structure

trace_record

Fields:

trace_id

source_entity

target_entity

relationship_type

timestamp

correlation_id

10.16 Relationship Types

Examples:

SUPPORTS

VALIDATES

GENERATED

REFERENCES

DERIVED_FROM

LINKED_TO

10.17 Lineage Architecture

Purpose:

Track decision ancestry.

Example:

Evidence A



Authority B



Opportunity C



Ranking D



Recommendation E

10.18 Lineage Service

Class:

LineageService

Responsibilities:

Create lineage

Traverse lineage

Visualize lineage

Retrieve lineage history

10.19 Lineage Storage

Table:

trace_records

Stores:

Entity Relationships

Decision Relationships

Workflow Relationships

10.20 Decision Traceability

Every recommendation stores:

Evidence References

Authority References

Ranking References

Formula Version

Timestamp

Correlation ID

10.21 Reproducibility Architecture

Purpose:

Recreate historical outcomes.

Required for:

Rankings

Recommendations

Authority Decisions

Evidence Decisions

Governance Decisions

10.22 Reproducibility Inputs

Required Data:

Historical Opportunity State

Historical Authority State

Historical Evidence State

Formula Version

Timestamp

Configuration Version

10.23 Snapshot Architecture

Purpose:

Preserve historical state.

Snapshot Types:

Ranking Snapshot

Recommendation Snapshot

Authority Snapshot

Evidence Snapshot

Governance Snapshot

10.24 Snapshot Structure

snapshot

Fields:

snapshot_id

entity_type

entity_id

version

created_at

snapshot_data

10.25 Replay Service

Class:

ReplayService

Responsibilities:

Load historical state

Execute replay

Compare results

Validate reproducibility

10.26 Reproducibility Validator

Class:

ReproducibilityValidator

Checks:

Input Integrity

Output Consistency

Version Consistency

Lineage Consistency

10.27 Reproducibility Example

Historical Inputs

↓

Replay Ranking Engine

↓

Replay Recommendation Engine

↓

Compare Results

↓

Reproducibility Report

10.28 Historical Reconstruction

Supported Workflows:

Authority Verification

Evidence Validation

Ranking Execution

Recommendation Generation

Governance Decisions

10.29 Audit Search

Supported Filters:

Actor

Entity

Action

Date Range

Correlation ID

Status

10.30 Trace Search

Supported Filters:

Source Entity

Target Entity

Relationship Type

Timestamp

Correlation ID

10.31 Governance Integration

ATR supports:

Compliance Reviews

Risk Reviews

Investigations

Dispute Resolution

Governance Audits

10.32 Database Tables

audit.audit_records

audit.trace_records

audit.snapshots

governance.decisions

rankings

recommendations

10.33 API Endpoints

GET

/api/v1/audit

Search audits

GET

/api/v1/audit/{id}

Audit details

GET

/api/v1/trace/{id}

Trace details

GET

/api/v1/lineage/{entity_id}

Lineage graph

POST

/api/v1/replay

Execute reproducibility replay

10.34 Observability

Metrics:

Audit Events

Trace Events

Lineage Records

Replay Executions

Reproducibility Success Rate

Audit Search Latency

10.35 Performance Requirements

Audit Write:

P95 < 50 ms

Trace Write:

P95 < 50 ms

Lineage Retrieval:

P95 < 500 ms

Replay Execution:

P95 < 3 s

Audit Search:

P95 < 500 ms

10.36 Error Handling

Failure Types:

Audit Failure

Trace Failure

Snapshot Failure

Replay Failure

Lineage Failure

Search Failure

All failures must generate audit events.

10.37 Testing Requirements

Unit Tests:

Audit Service

Trace Service

Lineage Service

Replay Service

Validator Service

Integration Tests:

Database Integration

Governance Integration

Recommendation Integration

Ranking Integration

Regression Tests:

Historical Reconstruction

Replay Accuracy

Lineage Consistency

10.38 Governance Rules

ATR-GOV-01

Audit records are immutable.

ATR-GOV-02

Critical workflows generate audit records.

ATR-GOV-03

Critical workflows generate trace records.

ATR-GOV-04

Lineage remains preserved.

ATR-GOV-05

Historical reconstruction remains possible.

ATR-GOV-06

Reproducibility remains measurable.

10.39 Architectural Constraints

AC-ATR-01

Audit records cannot be modified.

AC-ATR-02

Audit records cannot be deleted.

AC-ATR-03

Critical workflows require correlation IDs.

AC-ATR-04

Lineage records remain recoverable.

AC-ATR-05

Historical state remains preserved.

AC-ATR-06

Governance controls cannot bypass ATR.

10.40 Section Summary

The TODP ATR Architecture provides the implementation foundation for trust, accountability, and governance.

Business Event



Audit Record



Trace Record



Lineage Record



Historical Snapshot



Reproducibility Validation

The architecture ensures that every critical platform decision remains auditable, traceable, reproducible, and aligned with TODP's foundational trust principles.

IAD Progress

- 📄 Section 1 — Repository Structure & Implementation Foundation
 - 📄 Section 2 — Frontend Implementation Architecture (React)
 - 📄 Section 3 — Backend Implementation Architecture (FastAPI)
 - 📄 Section 4 — Database Schema Architecture & PostgreSQL Implementation
 - 📄 Section 5 — Retrieval Implementation Architecture (BM25 + pgvector + Hybrid Search)
 - 📄 Section 6 — Ranking Engine Implementation Architecture
 - 📄 Section 7 — Recommendation Engine Implementation Architecture
 - 📄 Section 8 — Authority Verification Implementation Architecture
 - 📄 Section 9 — Evidence Validation Implementation Architecture
 - 📄 Section 10 — Auditability, Traceability & Reproducibility Implementation Architecture

 - 📄 Section 11 — Event-Driven Architecture & Inter-Service Communication Implementation
- # IAD V1 — Section 11
- # Event-Driven Architecture & Inter-Service Communication Implementation

Document: Implementation Architecture Document (IAD V1)

Section: 11

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)

11.1 Purpose

This section defines the implementation architecture for event-driven communication inside TODP.

The architecture enables:

- Loose coupling
- Scalability
- Reliability
- Auditability
- Traceability

while preserving governance requirements.

11.2 Communication Philosophy

Not every service call should be synchronous.

Use synchronous communication when:

- Immediate response required
- User-facing operations
- Request-response workflows

Use asynchronous communication when:

- Background processing
- Notifications
- Audit generation
- Analytics
- Event propagation

11.3 Communication Model

TODP uses a hybrid architecture.

Synchronous



REST APIs

FastAPI

Asynchronous



Domain Events

Background Workers

Event Processing

11.4 Event Principles

EVT-01

Events are immutable.

EVT-02

Events are auditable.

EVT-03

Events are traceable.

EVT-04

Events are versioned.

EVT-05

Events contain correlation IDs.

EVT-06

Events support replay.

11.5 Event Architecture

Business Action

↓

Domain Event

↓

Event Publisher

↓

Event Processing

↓

Event Consumers

↓

Audit Generation

11.6 Event Module Structure

backend/app/infrastructure/messaging/

└─ publisher.py

- └─ consumer.py
- └─ event_bus.py
- └─ event_registry.py
- └─ event_models.py
- └─ event_handlers.py
- └─ event_audit.py

11.7 Event Categories

Category 1

Opportunity Events

Category 2

Authority Events

Category 3

Evidence Events

Category 4

Ranking Events

Category 5

Recommendation Events

Category 6

Governance Events

11.8 Opportunity Events

Examples:

OpportunityCreated

OpportunityUpdated

OpportunityArchived

OpportunityValidated

11.9 Authority Events

Examples:

AuthorityCreated

AuthorityVerified

AuthorityRejected

AuthorityReviewed

11.10 Evidence Events

Examples:

EvidenceCreated

EvidenceValidated

EvidenceRejected

EvidenceArchived

11.11 Ranking Events

Examples:

RankingStarted

RankingCompleted

RankingFailed

RankingVersionChanged

11.12 Recommendation Events

Examples:

RecommendationGenerated

RecommendationDelivered

RecommendationFailed

11.13 Governance Events

Examples:

PolicyApproved

PolicyUpdated

AuditGenerated

ComplianceReviewCompleted

11.14 Event Structure

Every event contains:

event_id

event_name

event_version

timestamp

correlation_id

source_service

payload

11.15 Event Example

OpportunityCreated

```
{  
  "event_id": "...",  
  "event_name": "OpportunityCreated",  
  "event_version": "v1",  
  "correlation_id": "...",  
  "timestamp": "...",  
  "payload": {}  
}
```

11.16 Event Publisher

Class:

EventPublisher

Responsibilities:

Publish events

Validate events

Attach metadata

Attach correlation IDs

11.17 Event Consumer

Class:

EventConsumer

Responsibilities:

Receive events

Validate events

Process events

Generate audit records

11.18 Event Bus

Class:

EventBus

Responsibilities:

Route events

Manage subscriptions

Support event delivery

Support event replay

11.19 Correlation ID Propagation

Request



Retrieval



Ranking



Recommendation



Audit



Events

All share same correlation_id.

11.20 Event Replay

Purpose:

Support reproducibility.

Support debugging.

Support investigations.

Replay uses:

Stored Events



Replay Engine



Validation

11.21 Event Storage

Table:

event_store

Fields:

event_id

event_name

event_version

payload

timestamp

correlation_id

11.22 Event Versioning

Rules:

Events are never modified.

New schema



New version

Example:

OpportunityCreatedV1

OpportunityCreatedV2

11.23 Event Reliability

Requirements:

At-Least-Once Delivery

Retry Support

Failure Recovery

Dead Letter Queue Support

11.24 Dead Letter Queue

Purpose:

Capture failed events.

Store:

event

error

timestamp

retry_count

11.25 Event Auditability

Every event generates:

Audit Record

↓

Trace Record



Lineage Record

11.26 Event Observability

Metrics:

Published Events

Consumed Events

Failed Events

Replay Count

Retry Count

Processing Time

11.27 Event Performance Targets

Publish Event:

P95 < 50 ms

Consume Event:

P95 < 100 ms

Replay Event:

P95 < 500 ms

11.28 Testing Requirements

Unit Tests

Publishers

Consumers

Event Validation

Integration Tests

Event Flow

Retry Flow

Replay Flow

Performance Tests

High Event Volume

Consumer Scaling

11.29 Governance Rules

EVT-GOV-01

Events are immutable.

EVT-GOV-02

Events are versioned.

EVT-GOV-03

Events are auditable.

EVT-GOV-04

Events are traceable.

EVT-GOV-05

Events support replay.

11.30 Architectural Constraints

AC-EVT-01

Events cannot be modified after publication.

AC-EVT-02

Events require correlation IDs.

AC-EVT-03

Events require versioning.

AC-EVT-04

Critical events must be stored.

AC-EVT-05

Governance controls cannot bypass event auditing.

11.31 Section Summary

The TODP Event Architecture provides a scalable and auditable communication framework.

Business Action



Domain Event



Event Bus



Consumers



Audit & Traceability

The architecture enables loose coupling, scalability, observability, reproducibility, and governance-compliant communication across the platform.

IAD Progress

☑ Sections 1–10 Complete

☑ Section 11 — Event-Driven Architecture & Inter-Service Communication

☑ Section 12 — Authentication & Authorization Implementation Architecture

IAD V1 — Section 12

Authentication & Authorization Implementation Architecture

Document: Implementation Architecture Document (IAD V1)

Section: 12

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)

- ADD V1 (Frozen)

- DDD V1 (Frozen)
- SDD V1 (Frozen)

12.1 Purpose

This section defines the Authentication and Authorization implementation architecture for TODP.

The security architecture is responsible for:

- Identity verification
- User authentication
- Session management
- Access control
- Role management
- Permission enforcement
- Security auditing

Authentication answers:

"Who are you?"

Authorization answers:

"What are you allowed to do?"

12.2 Security Philosophy

Security is a platform capability.

Security must be:

- Centralized
- Auditable
- Traceable
- Observable
- Governed

Security controls cannot be bypassed.

12.3 Security Principles

SEC-01

Every user must be authenticated.

SEC-02

Every request must be authorized.

SEC-03

Least privilege applies.

SEC-04

Security actions are auditable.

SEC-05

Security events are traceable.

SEC-06

Permissions are role-based.

12.4 Security Architecture

User



Authentication

↓

JWT Validation

↓

Authorization

↓

Permission Check

↓

Business Service

12.5 Security Module Structure

backend/app/security/

├─ auth/

| └─ login_service.py

| └─ token_service.py

| └─ password_service.py

|

```
└─ authorization/
|   └─ permission_service.py
|   └─ role_service.py
|   └─ policy_service.py
|
└─ middleware/
|   └─ auth_middleware.py
|   └─ security_middleware.py
|
└─ audit/
    └─ security_audit.py
```

12.6 Authentication Flow

Login Request

↓

Credential Validation

↓

User Verification

↓

JWT Generation



Token Returned



Authenticated Session

12.7 User Entity

users

Fields:

id

email

password_hash

status

role

created_at

updated_at

last_login

12.8 User Status

ACTIVE

INACTIVE

LOCKED

SUSPENDED

PENDING

12.9 Authentication Service

Class:

AuthenticationService

Responsibilities:

Validate credentials

Generate tokens

Verify users

Manage sessions

Record security events

12.10 Password Management

Requirements:

Strong passwords

Password hashing

Password rotation support

Password reset support

Password history support

12.11 Password Storage

Passwords are never stored in plaintext.

Store:

password_hash

Algorithm:

bcrypt

or equivalent secure algorithm

12.12 JWT Architecture

Access Token

Short-lived

Refresh Token

Long-lived

Benefits:

Scalability

Stateless APIs

Security

12.13 JWT Payload

Contains:

user_id

email

role

permissions

issued_at

expires_at

token_id

12.14 Token Lifecycle

Login



Access Token Issued



API Access



Expiration



Refresh



New Token

12.15 Authorization Architecture

Authentication



Role Resolution



Permission Resolution



Policy Validation



Access Decision

12.16 Role-Based Access Control (RBAC)

Primary Model:

RBAC

Role



Permissions



Actions

12.17 System Roles

Role 1

User

Role 2

Reviewer

Role 3

Administrator

Role 4

Governance Administrator

Role 5

Security Administrator

Role 6

System Administrator

12.18 Permission Structure

Examples:

opportunity.read

opportunity.create

authority.verify

evidence.validate

ranking.execute

recommendation.generate

audit.read

policy.manage

12.19 Authorization Service

Class:

AuthorizationService

Responsibilities:

Resolve permissions

Validate access

Evaluate policies

Generate access decisions

12.20 API Protection

Every protected endpoint requires:

Valid JWT

↓

Role Check

↓

Permission Check

↓

Access Granted

12.21 Example Endpoint Security

POST

/api/v1/authorities/{id}/verify

Required Permission:

authority.verify

GET

/api/v1/audit

Required Permission:

audit.read

12.22 Security Middleware

Responsibilities:

Token Validation

Request Authentication

Permission Loading

Security Logging

Correlation ID Propagation

12.23 Session Management

Store:

Session ID

User ID

Login Time

Last Activity

Status

Purpose:

Session tracking

Security monitoring

Forced logout support

12.24 Security Audit Events

Examples:

LOGIN

LOGOUT

TOKEN_REFRESH

ACCESS_DENIED

PASSWORD_RESET

ROLE_CHANGED

PERMISSION_CHANGED

12.25 Security Audit Record

Fields:

event_id

user_id

event_type

timestamp

ip_address

correlation_id

metadata

12.26 Failed Login Protection

Controls:

Rate Limiting

Temporary Lockout

Audit Logging

Suspicious Activity Detection

12.27 Account Recovery

Supported Functions:

Forgot Password

Password Reset

Account Unlock

Identity Verification

12.28 Security Database Tables

users

roles

permissions

role_permissions

user_roles

sessions

security_events

audit_records

12.29 API Security Dependencies

FastAPI Dependency Injection:

get_current_user()

require_role()

require_permission()

validate_token()

12.30 Observability

Metrics:

Login Count

Failed Logins

Token Refresh Count

Access Denials

Role Changes

Permission Changes

12.31 Performance Requirements

Token Validation:

P95 < 20 ms

Permission Check:

P95 < 10 ms

Authentication:

P95 < 100 ms

Authorization:

P95 < 20 ms

12.32 Error Handling

Failure Types:

Invalid Credentials

Expired Token

Invalid Token

Permission Denied

Role Missing

Account Locked

All failures generate security audit events.

12.33 Testing Requirements

Unit Tests:

Authentication Service

Authorization Service

Token Service

Permission Service

Role Service

Integration Tests:

Login Flow

Token Refresh Flow

Permission Validation

Protected Endpoints

Security Tests:

Privilege Escalation

Token Tampering

Unauthorized Access

Session Hijacking Simulation

12.34 Governance Rules

SEC-GOV-01

All users must authenticate.

SEC-GOV-02

All protected endpoints require authorization.

SEC-GOV-03

Security events must be audited.

SEC-GOV-04

Roles must be governed.

SEC-GOV-05

Permissions must be traceable.

SEC-GOV-06

Security logs must be preserved.

12.35 Architectural Constraints

AC-SEC-01

Authentication cannot be bypassed.

AC-SEC-02

Authorization cannot be bypassed.

AC-SEC-03

Passwords are never stored in plaintext.

AC-SEC-04

JWTs must be validated on every request.

AC-SEC-05

Security events must be auditable.

AC-SEC-06

Least privilege remains mandatory.

12.36 Section Summary

The TODP Authentication & Authorization Architecture provides centralized identity and access management.

User



Authentication



JWT Validation



Role Resolution



Permission Validation



Business Service



Security Audit

The architecture ensures secure access control while preserving traceability, auditability, observability, and governance compliance.

IAD Progress

- 🔗 Section 1 — Repository Structure & Implementation Foundation
- 🔗 Section 2 — Frontend Implementation Architecture (React)
- 🔗 Section 3 — Backend Implementation Architecture (FastAPI)
- 🔗 Section 4 — Database Schema Architecture & PostgreSQL Implementation
- 🔗 Section 5 — Retrieval Implementation Architecture (BM25 + pgvector + Hybrid Search)
- 🔗 Section 6 — Ranking Engine Implementation Architecture
- 🔗 Section 7 — Recommendation Engine Implementation Architecture
- 🔗 Section 8 — Authority Verification Implementation Architecture
- 🔗 Section 9 — Evidence Validation Implementation Architecture
- 🔗 Section 10 — Auditability, Traceability & Reproducibility Implementation Architecture
- 🔗 Section 11 — Event-Driven Architecture & Inter-Service Communication Implementation
- 🔗 Section 12 — Authentication & Authorization Implementation Architecture

▣ Section 13 — API Gateway & External Interface Implementation Architecture

IAD V1 — Section 13

API Gateway & External Interface Implementation Architecture

Document: Implementation Architecture Document (IAD V1)

Section: 13

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)

13.1 Purpose

This section defines the API Gateway and External Interface implementation architecture for TODP.

The API layer is responsible for:

- External access

- Request routing
- Authentication enforcement
- Authorization enforcement
- Rate limiting
- Request validation
- Response standardization
- API observability

The API Gateway acts as the controlled entry point into the platform.

13.2 API Philosophy

Clients interact with APIs.

Clients do not interact directly with:

- Databases
- Internal Services
- Event Infrastructure
- Audit Infrastructure

All access must occur through governed APIs.

13.3 API Principles

API-01

APIs are versioned.

API-02

APIs are documented.

API-03

APIs are authenticated.

API-04

APIs are authorized.

API-05

APIs are observable.

API-06

APIs are auditable.

13.4 API Architecture

Frontend



API Gateway



Authentication



Authorization



Application Services



Domain Services

13.5 Gateway Responsibilities

Request Routing

Authentication

Authorization

Rate Limiting

Request Validation

Response Formatting

Logging

Telemetry

13.6 API Structure

/api/v1/

└─ auth

└─ users

└─ opportunities

- └─ authorities
- └─ evidence
- └─ retrieval
- └─ rankings
- └─ recommendations
- └─ creation
- └─ audit
- └─ governance

13.7 API Versioning Strategy

Version Format:

/api/v1/

Future:

/api/v2/

/api/v3/

Rules:

Breaking changes require new versions.

Non-breaking changes remain within version.

13.8 REST Design Standards

Use nouns.

Good:

GET /opportunities

GET /authorities

GET /evidence

Avoid:

GET /getOpportunities

GET /fetchAuthorities

13.9 HTTP Method Standards

GET

Read

POST

Create

PUT

Replace

PATCH

Update

DELETE

Soft Delete

13.10 Authentication Integration

Every protected endpoint requires:

Authorization Header



JWT Validation



User Resolution



Permission Validation



Request Processing

13.11 Authorization Integration

Permission validation occurs before business execution.

Example:

authority.verify

evidence.validate

ranking.execute

recommendation.generate

13.12 Rate Limiting

Purpose:

Protect platform stability.

Prevent abuse.

Prevent automated attacks.

13.13 Rate Limiting Rules

Anonymous Users

100 requests/hour

Authenticated Users

1000 requests/hour

Administrative APIs

Governed separately

13.14 Request Validation

Validation occurs before service execution.

Validation Types:

Schema Validation

Data Validation

Security Validation

Permission Validation

13.15 Response Standardization

Every response follows a common structure.

Response:

success

data

errors

timestamp

correlation_id

13.16 Success Response Example

```
{  
  "success": true,  
  "data": {},  
  "timestamp": "...",  
  "correlation_id": "..."  
}
```

13.17 Error Response Example

```
{  
  "success": false,  
  "errors": [],  
  "timestamp": "...",  
  "correlation_id": "..."  
}
```

13.18 Correlation ID Management

Every request receives:

correlation_id

Purpose:

Traceability

Auditability

Observability

Debugging

13.19 API Documentation

Documentation Standard:

OpenAPI

Swagger

Generated automatically by FastAPI.

13.20 Public APIs

Examples:

Opportunity Discovery

Opportunity Details

Authority Information

Evidence Information

Authentication may still apply.

13.21 Internal APIs

Examples:

Ranking Execution

Evidence Validation

Authority Verification

Governance Operations

Audit Operations

Restricted access.

13.22 Opportunity APIs

GET

/opportunities

GET

/opportunities/{id}

POST

/opportunities

PATCH

/opportunities/{id}

13.23 Authority APIs

GET

/authorities

GET

/authorities/{id}

POST

/authorities

POST

/authorities/{id}/verify

13.24 Evidence APIs

GET

/evidence

GET

/evidence/{id}

POST

/evidence

POST

/evidence/{id}/validate

13.25 Retrieval APIs

POST

/retrieval/search

Input:

query

filters

user_context

Output:

candidate_set

13.26 Ranking APIs

POST

/rankings/execute

Input:

candidate_set

Output:

ranked_results

13.27 Recommendation APIs

POST

/recommendations/generate

Input:

ranking_id

Output:

recommendations

GET

/recommendations/{id}

13.28 Audit APIs

GET

/audit

GET

/audit/{id}

GET

/trace/{id}

GET

/lineage/{id}

13.29 Governance APIs

GET

/policies

POST

/policies

GET

/decisions

POST

/decisions

13.30 API Error Categories

400

Validation Error

401

Authentication Error

403

Authorization Error

404

Resource Not Found

409

Conflict

500

System Error

13.31 API Security Controls

JWT Validation

Permission Validation

Rate Limiting

Input Validation

Security Logging

Correlation Tracking

13.32 API Observability

Metrics:

Request Count

Error Count

Latency

Success Rate

Rate Limit Violations

Authentication Failures

Authorization Failures

13.33 API Auditability

Every request generates:

API Event

↓

Audit Record

↓

Trace Record

↓

Telemetry Event

13.34 API Performance Requirements

GET Requests:

P95 < 200 ms

POST Requests:

P95 < 500 ms

Search Requests:

P95 < 2 s

Recommendation Requests:

P95 < 500 ms

13.35 API Testing Requirements

Unit Tests:

Route Validation

Schema Validation

Permission Validation

Integration Tests:

Authentication

Authorization

Database Integration

Service Integration

Contract Tests:

Request Schemas

Response Schemas

Version Compatibility

13.36 Governance Rules

API-GOV-01

All APIs are versioned.

API-GOV-02

All APIs are documented.

API-GOV-03

Protected APIs require authentication.

API-GOV-04

Protected APIs require authorization.

API-GOV-05

All APIs are observable.

API-GOV-06

All APIs are auditable.

13.37 Architectural Constraints

AC-API-01

Direct database access is prohibited.

AC-API-02

All access occurs through APIs.

AC-API-03

Breaking changes require versioning.

AC-API-04

Correlation IDs are mandatory.

AC-API-05

Security validation is mandatory.

AC-API-06

Governance controls cannot be bypassed.

13.38 Section Summary

The TODP API Gateway Architecture provides a secure, governed, observable, and auditable interface layer for all platform interactions.

Client



API Gateway



Authentication



Authorization



Application Services



Domain Services



Persistence

The architecture ensures controlled access while preserving traceability, auditability, reproducibility, and governance compliance.

IAD Progress

- 📄 Section 1 — Repository Structure & Implementation Foundation
- 📄 Section 2 — Frontend Implementation Architecture (React)
- 📄 Section 3 — Backend Implementation Architecture (FastAPI)
- 📄 Section 4 — Database Schema Architecture & PostgreSQL Implementation
- 📄 Section 5 — Retrieval Implementation Architecture (BM25 + pgvector + Hybrid Search)
- 📄 Section 6 — Ranking Engine Implementation Architecture
- 📄 Section 7 — Recommendation Engine Implementation Architecture
- 📄 Section 8 — Authority Verification Implementation Architecture
- 📄 Section 9 — Evidence Validation Implementation Architecture
- 📄 Section 10 — Auditability, Traceability & Reproducibility Implementation Architecture
- 📄 Section 11 — Event-Driven Architecture & Inter-Service Communication Implementation
- 📄 Section 12 — Authentication & Authorization Implementation Architecture
- 📄 Section 13 — API Gateway & External Interface Implementation Architecture

- 📄 Section 14 — Redis Caching Architecture & Performance Optimization Implementation

IAD V1 — Section 14

Redis Caching Architecture & Performance Optimization Implementation

Document: Implementation Architecture Document (IAD V1)

Section: 14

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)

14.1 Purpose

This section defines the Redis caching architecture and performance optimization implementation for TODP.

The caching layer is responsible for:

- Reducing database load
- Reducing API latency
- Improving search performance
- Improving recommendation performance
- Improving scalability
- Improving user experience

Redis is a performance layer.

Redis is not a source of truth.

14.2 Caching Philosophy

Source of Truth

↓

PostgreSQL

↓

Cache Layer

↓

Application Services

The platform must remain operational even when cache is unavailable.

14.3 Caching Principles

CACHE-01

Database remains authoritative.

CACHE-02

Cache remains disposable.

CACHE-03

Cache failures must not cause data loss.

CACHE-04

Cache contents must be reproducible.

CACHE-05

Cache invalidation must be controlled.

CACHE-06

Cache activity must be observable.

14.4 Cache Architecture

Client



API



Application Service



Redis Cache



PostgreSQL

Cache Hit



Return Response

Cache Miss

↓

Database Query

↓

Cache Populate

↓

Return Response

14.5 Technology Stack

Cache Engine:

Redis

Client Library:

redis-py

Deployment:

Containerized

Persistence:

Optional

Primary Role:

Performance Optimization

14.6 Cache Module Structure

backend/app/infrastructure/cache/

├─ redis_client.py

├─ cache_service.py

├─ cache_keys.py

├─ cache_policies.py

├─ cache_metrics.py

└─ cache_invalidation.py

14.7 Cache Categories

Category 1

Reference Data Cache

Category 2

Search Cache

Category 3

Recommendation Cache

Category 4

Authority Cache

Category 5

Evidence Cache

Category 6

Configuration Cache

14.8 Reference Data Cache

Examples:

Categories

Regions

States

Districts

Static Metadata

Purpose:

Reduce repeated lookups.

14.9 Search Cache

Stores:

Search Results

Candidate Sets

Query Responses

Purpose:

Reduce retrieval latency.

14.10 Recommendation Cache

Stores:

Recommendation Results

Recommendation Metadata

Explanation Payloads

Purpose:

Reduce recommendation latency.

14.11 Authority Cache

Stores:

Authority Records

Authority Verification Status

Authority Metadata

Purpose:

Reduce authority lookup time.

14.12 Evidence Cache

Stores:

Evidence Metadata

Evidence Relationships

Evidence References

Purpose:

Reduce evidence retrieval latency.

14.13 Configuration Cache

Stores:

Application Configuration

Feature Flags

System Settings

Governance Settings

Purpose:

Fast configuration access.

14.14 Cache Key Standards

Format:

todp:{domain}:{identifier}

Examples:

todp:opportunity:123

todp:authority:456

todp:evidence:789

todp:search:query_hash

14.15 Search Cache Keys

Examples:

todp:search:scholarship

todp:search:online_mba

todp:search:government_grant

Purpose:

Store retrieval results.

14.16 Recommendation Cache Keys

Examples:

todp:recommendation:user_123

todp:recommendation:ranking_456

Purpose:

Store recommendation outputs.

14.17 Cache Access Layer

Class:

CacheService

Responsibilities:

Read Cache

Write Cache

Invalidate Cache

Monitor Cache

14.18 Cache Operations

GET

Retrieve Cache

SET

Store Cache

DELETE

Remove Cache

EXPIRE

Set TTL

14.19 Cache Strategy

Primary Strategy:

Cache Aside Pattern

Workflow:

Request

↓

Cache Check



Miss



Database Query



Cache Store



Response

14.20 Cache TTL Policy

Reference Data

24 Hours

Authority Data

1 Hour

Evidence Data

1 Hour

Search Results

15 Minutes

Recommendations

15 Minutes

Configuration

30 Minutes

14.21 Cache Invalidation

Triggers:

Data Update

Data Delete

Authority Verification

Evidence Validation

Configuration Change

Policy Change

14.22 Cache Invalidation Service

Class:

CacheInvalidationService

Responsibilities:

Locate Keys

Remove Keys

Refresh Entries

Track Invalidations

14.23 Search Cache Flow

Search Request



Cache Lookup



Hit



Return Results

OR



Miss



Hybrid Retrieval



Store Results



Return Results

14.24 Recommendation Cache Flow

Recommendation Request



Cache Lookup



Hit



Return Recommendation

OR



Miss



Generate Recommendation



Store Result



Return Recommendation

14.25 Cache Consistency Rules

Database Update

↓

Invalidate Cache

↓

Future Requests Rebuild Cache

Consistency Priority:

Correctness

>

Performance

14.26 Redis Data Structures

Strings

Reference Data

Hashes

Entity Objects

Sets

Relationships

Sorted Sets

Ranked Data

14.27 Distributed Caching

Future Capability:

Multiple Redis Instances

Purpose:

Horizontal Scalability

High Availability

Improved Throughput

14.28 Cache Security

Requirements:

Authentication

Network Isolation

TLS Support

Access Control

Audit Logging

14.29 Cache Auditability

Events Logged:

Cache Writes

Cache Deletes

Cache Invalidations

Cache Failures

Configuration Changes

14.30 Cache Observability

Metrics:

Cache Hits

Cache Misses

Hit Ratio

Evictions

Latency

Memory Usage

Invalidation Count

14.31 Performance Targets

Cache Read:

P95 < 5 ms

Cache Write:

P95 < 10 ms

Cache Invalidation:

P95 < 20 ms

Search Cache Response:

P95 < 50 ms

Recommendation Cache Response:

P95 < 50 ms

14.32 Failure Handling

Failure Types:

Redis Unavailable

Network Failure

Cache Corruption

Memory Pressure

Timeout

Fallback:

Application

↓

Database

↓

Response

Platform remains operational.

14.33 Testing Requirements

Unit Tests:

Cache Service

Key Generation

TTL Rules

Invalidation Logic

Integration Tests:

Redis Integration

Search Integration

Recommendation Integration

Authority Integration

Performance Tests:

Hit Rate Validation

Load Testing

Stress Testing

14.34 Governance Rules

CACHE-GOV-01

Redis is not a source of truth.

CACHE-GOV-02

Cache entries are disposable.

CACHE-GOV-03

Cache invalidation is mandatory.

CACHE-GOV-04

Cache activity is observable.

CACHE-GOV-05

Cache failures do not stop business operations.

CACHE-GOV-06

Cache configuration is governed.

14.35 Architectural Constraints

AC-CACHE-01

PostgreSQL remains authoritative.

AC-CACHE-02

Redis cannot store exclusive business state.

AC-CACHE-03

Critical data must exist in PostgreSQL.

AC-CACHE-04

Cache must support invalidation.

AC-CACHE-05

Cache failures must degrade gracefully.

AC-CACHE-06

Governance controls cannot be bypassed.

14.36 Section Summary

The TODP Redis Architecture provides a high-performance caching layer that improves responsiveness and scalability while preserving data integrity.

Application Request



Redis Cache



Cache Hit

OR



PostgreSQL



Cache Refresh



Response

The architecture ensures performance optimization without compromising auditability, traceability, reproducibility, governance, or trust.

IAD Progress

📄 Section 1 — Repository Structure & Implementation Foundation

📄 Section 2 — Frontend Implementation Architecture (React)

📄 Section 3 — Backend Implementation Architecture (FastAPI)

📄 Section 4 — Database Schema Architecture & PostgreSQL Implementation

- 🔗 Section 5 — Retrieval Implementation Architecture (BM25 + pgvector + Hybrid Search)
- 🔗 Section 6 — Ranking Engine Implementation Architecture
- 🔗 Section 7 — Recommendation Engine Implementation Architecture
- 🔗 Section 8 — Authority Verification Implementation Architecture
- 🔗 Section 9 — Evidence Validation Implementation Architecture
- 🔗 Section 10 — Auditability, Traceability & Reproducibility Implementation Architecture
- 🔗 Section 11 — Event-Driven Architecture & Inter-Service Communication Implementation
- 🔗 Section 12 — Authentication & Authorization Implementation Architecture
- 🔗 Section 13 — API Gateway & External Interface Implementation Architecture
- 🔗 Section 14 — Redis Caching Architecture & Performance Optimization Implementation
- 🔗 Section 15 — Observability Implementation Architecture (OpenTelemetry, Prometheus & Grafana)

IAD V1 — Section 15

Observability Implementation Architecture (OpenTelemetry, Prometheus & Grafana)

Document: Implementation Architecture Document (IAD V1)

Section: 15

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)

15.1 Purpose

This section defines the implementation architecture for observability within TODP.

Observability is responsible for:

- Monitoring platform health
- Monitoring business workflows
- Monitoring performance
- Monitoring reliability
- Monitoring security events
- Monitoring governance compliance

Observability provides visibility into the entire platform.

15.2 Observability Philosophy

If a system cannot be observed, it cannot be trusted.

Observability supports:

- Reliability
- Scalability

- Governance
- Auditability
- Traceability
- Operational Excellence

Every critical workflow must be observable.

15.3 Observability Principles

OBS-01

Critical workflows generate telemetry.

OBS-02

Critical services generate metrics.

OBS-03

Critical requests generate traces.

OBS-04

Critical failures generate alerts.

OBS-05

Observability data remains searchable.

OBS-06

Observability remains governed.

15.4 Technology Stack

Distributed Tracing:

OpenTelemetry

Metrics:

Prometheus

Dashboards:

Grafana

Application Logs:

Structured JSON Logging

Backend:

FastAPI Instrumentation

Frontend:

React Instrumentation

15.5 Observability Architecture

Application

↓

OpenTelemetry

↓

Metrics

Traces

Logs

↓

Prometheus

↓

Grafana



Dashboards & Alerts

15.6 Observability Components

Component 1

Metrics Collection

Component 2

Distributed Tracing

Component 3

Structured Logging

Component 4

Alerting

Component 5

Dashboards

Component 6

Operational Analytics

15.7 Metrics Architecture

Purpose:

Quantitative platform monitoring.

Examples:

Request Count

Error Count

Latency

Throughput

Cache Hit Rate

Search Volume

Recommendation Volume

15.8 Metric Categories

Infrastructure Metrics

Application Metrics

Business Metrics

Security Metrics

Governance Metrics

15.9 Infrastructure Metrics

CPU Usage

Memory Usage

Disk Usage

Network Throughput

Container Health

Database Connections

Redis Connections

15.10 Application Metrics

API Requests

Response Times

Error Rates

Authentication Events

Authorization Failures

Queue Depth

Cache Performance

15.11 Business Metrics

Search Requests

Opportunity Views

Ranking Executions

Recommendation Executions

Authority Verifications

Evidence Validations

15.12 Governance Metrics

Audit Events

Trace Events

Replay Executions

Compliance Reviews

Policy Updates

Governance Decisions

15.13 Distributed Tracing

Purpose:

Track complete request lifecycle.

Example:

User Request

↓

API

↓

Retrieval

↓

Ranking



Recommendation



Response

15.14 Trace Structure

trace_id

span_id

parent_span_id

correlation_id

service_name

operation_name

timestamp

duration

15.15 Trace Propagation

All services propagate:

trace_id

correlation_id

request_context

Purpose:

End-to-end visibility.

15.16 Backend Instrumentation

Instrument:

API Requests

Database Queries

Redis Calls

Ranking Execution

Recommendation Execution

Authority Verification

Evidence Validation

15.17 Frontend Instrumentation

Instrument:

Page Views

Search Events

User Actions

API Calls

Client Errors

Performance Metrics

15.18 Logging Architecture

Format:

Structured JSON

Example Fields:

timestamp

service

level

message

correlation_id

trace_id

user_id

metadata

15.19 Log Levels

DEBUG

Development

INFO

Normal Operations

WARNING

Potential Issues

ERROR

Failures

CRITICAL

System Threats

15.20 Logging Standards

Every log must include:

Timestamp

Service Name

Correlation ID

Severity

Message

Context

15.21 Dashboard Categories

Operational Dashboard

Application Dashboard

Business Dashboard

Security Dashboard

Governance Dashboard

15.22 Operational Dashboard

Displays:

System Health

CPU

Memory

Storage

Container Status

Network Status

15.23 Application Dashboard

Displays:

Request Volume

Error Rates

Latency

Database Performance

Redis Performance

API Performance

15.24 Business Dashboard

Displays:

Search Activity

Ranking Activity

Recommendation Activity

Authority Activity

Evidence Activity

User Activity

15.25 Security Dashboard

Displays:

Login Activity

Failed Logins

Authorization Failures

Role Changes

Permission Changes

Security Events

15.26 Governance Dashboard

Displays:

Audit Volume

Trace Volume

Lineage Activity

Replay Activity

Policy Changes

Compliance Reviews

15.27 Alerting Architecture

Purpose:

Notify operational teams of issues.

Alert Types:

Infrastructure

Application

Security

Governance

Business

15.28 Infrastructure Alerts

CPU > Threshold

Memory > Threshold

Disk Usage > Threshold

Container Failure

Database Unavailable

Redis Unavailable

15.29 Application Alerts

High Latency

High Error Rate

API Failures

Queue Backlog

Service Failure

15.30 Security Alerts

Repeated Login Failures

Privilege Escalation Attempts

Unauthorized Access Attempts

Suspicious Activity

Token Abuse

15.31 Governance Alerts

Audit Failure

Trace Failure

Replay Failure

Missing Correlation ID

Policy Violation

15.32 Correlation Framework

Every observable event includes:

correlation_id

Purpose:

Connect logs

Connect metrics

Connect traces

Connect audits

15.33 OpenTelemetry Implementation

Instrumented Components:

FastAPI

SQLAlchemy

Redis

HTTP Clients

Background Workers

Event

IAD V1 — Section 16

Testing & Quality Assurance Implementation Architecture

Document: Implementation Architecture Document (IAD V1)

Section: 16

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)

16.1 Purpose

This section defines the testing and quality assurance implementation architecture for TODP.

The QA architecture is responsible for:

- Functional validation
- Integration validation
- Performance validation
- Security validation

- Governance validation
- Regression prevention

Testing is a platform-wide responsibility.

Quality cannot be inspected into a system after development.

Quality must be built into development.

16.2 Testing Philosophy

Every important platform capability must be:

Implemented



Tested



Validated



Released

No capability is considered complete without testing.

16.3 Testing Principles

TEST-01

Testing is automated by default.

TEST-02

Critical workflows require coverage.

TEST-03

Regression testing is mandatory.

TEST-04

Security testing is mandatory.

TEST-05

Governance testing is mandatory.

TEST-06

Production deployments require validation.

16.4 Testing Pyramid

E2E Tests

↑

Integration Tests

↑

Unit Tests

Unit tests provide the majority of coverage.

16.5 Testing Architecture

Developer Code



Unit Tests



Integration Tests



Contract Tests



Security Tests



Performance Tests



E2E Tests



Release Decision

16.6 Test Repository Structure

tests/

- |— unit/
- |— integration/
- |— contract/
- |— performance/
- |— security/
- |— governance/
- |— e2e/
- └─ fixtures/

16.7 Unit Testing

Purpose:

Validate isolated components.

Targets:

Services

Validators

Utilities

Calculators

Repositories

Policies

16.8 Unit Testing Tools

Backend:

Pytest

Frontend:

Vitest

React Testing Library

16.9 Unit Test Coverage Targets

Domain Services:

≥ 90%

Business Rules:

≥ 95%

Validators:

≥ 95%

Utility Functions:

≥ 90%

16.10 Integration Testing

Purpose:

Validate interaction between components.

Examples:

API ↔ Service

Service ↔ Database

Service ↔ Redis

Service ↔ Event Bus

Service ↔ Governance Layer

16.11 Integration Test Scenarios

Opportunity Creation

Authority Verification

Evidence Validation

Ranking Execution

Recommendation Generation

Audit Generation

16.12 Contract Testing

Purpose:

Validate API compatibility.

Focus:

Request Schemas

Response Schemas

Version Compatibility

Breaking Change Detection

16.13 API Contract Validation

Verify:

Required Fields

Optional Fields

Response Structures

Error Structures

Status Codes

16.14 End-to-End Testing

Purpose:

Validate complete workflows.

Example:

User Search

↓

Retrieval

↓

Ranking

↓

Recommendation



Explanation



Response

16.15 E2E Test Scenarios

Discovery Journey

Physical Opportunity Journey

Online Opportunity Journey

Authority Verification Journey

Evidence Validation Journey

Governance Review Journey

16.16 Performance Testing

Purpose:

Validate scalability and latency.

Areas:

API Performance

Search Performance

Ranking Performance

Recommendation Performance

Database Performance

Cache Performance

16.17 Performance Targets

API Latency:

P95 < 300 ms

Search:

P95 < 2 s

Ranking:

P95 < 1 s

Recommendation:

P95 < 500 ms

Cache Reads:

P95 < 5 ms

16.18 Load Testing

Purpose:

Validate expected production load.

Scenarios:

Concurrent Searches

Concurrent Recommendations

Concurrent API Requests

Concurrent User Sessions

16.19 Stress Testing

Purpose:

Identify breaking points.

Examples:

10x Traffic

Large Candidate Sets

Database Pressure

Cache Failure Simulation

16.20 Security Testing

Purpose:

Validate security controls.

Areas:

Authentication

Authorization

Input Validation

Token Validation

Session Security

Privilege Escalation

16.21 Security Test Cases

Invalid JWT

Expired JWT

Permission Bypass

Role Escalation

Injection Attacks

Unauthorized Access

16.22 Governance Testing

Purpose:

Validate trust architecture.

Areas:

Auditability

Traceability

Reproducibility

Lineage

Compliance

16.23 Governance Test Cases

Audit Generation

Trace Generation

Lineage Preservation

Replay Validation

Correlation Tracking

Policy Enforcement

16.24 Retrieval Testing

Validate:

BM25 Retrieval

Vector Retrieval

Hybrid Retrieval

Fusion Logic

Search Reproducibility

16.25 Ranking Testing

Validate:

Money Cost Calculation

Time Cost Calculation

Distance Cost Calculation

Tie Resolution

Ranking Reproducibility

16.26 Recommendation Testing

Validate:

Authority Attachment

Evidence Attachment

Explanation Generation

Recommendation Reproducibility

Recommendation Traceability

16.27 Authority Testing

Validate:

Authority Verification

Confidence Calculation

Authority Lifecycle

Authority History

Authority Traceability

16.28 Evidence Testing

Validate:

Evidence Validation

Evidence Versioning

Evidence Lineage

Evidence History

Evidence Traceability

16.29 Database Testing

Validate:

Schema Integrity

Migration Execution

Indexes

Constraints

Referential Integrity

Rollback Procedures

16.30 Cache Testing

Validate:

Cache Hits

Cache Misses

TTL Policies

Cache Invalidation

Redis Failure Recovery

16.31 Observability Testing

Validate:

Metrics Generation

Trace Generation

Log Generation

Alert Generation

Dashboard Accuracy

16.32 Test Data Management

Requirements:

Versioned

Repeatable

Isolated

Auditable

Reusable

16.33 Test Environments

Development



Test



Stage



Production

Each environment maintains independent datasets.

16.34 CI Testing Pipeline

Code Commit



Unit Tests



Integration Tests



Contract Tests



Security Tests



Build



Deploy

16.35 Release Gates

Release blocked if:

Critical Tests Fail

Security Tests Fail

Governance Tests Fail

Performance Thresholds Fail

Coverage Thresholds Fail

16.36 Defect Classification

Critical

High

Medium

Low

All defects tracked.

16.37 Quality Metrics

Code Coverage

Defect Density

Regression Failures

Build Success Rate

Deployment Success Rate

Performance Compliance

16.38 Testing Observability

Metrics:

Test Duration

Test Success Rate

Failure Count

Coverage

Regression Count

Environment Health

16.39 Governance Rules

TEST-GOV-01

Critical workflows require automated testing.

TEST-GOV-02

Regression testing is mandatory.

TEST-GOV-03

Governance controls require validation.

TEST-GOV-04

Security controls require validation.

TEST-GOV-05

Production releases require test approval.

TEST-GOV-06

Test results remain auditable.

16.40 Architectural Constraints

AC-TEST-01

Untested code cannot be released.

AC-TEST-02

Critical workflows require coverage.

AC-TEST-03

Security failures block release.

AC-TEST-04

Governance failures block release.

AC-TEST-05

Regression failures block release.

AC-TEST-06

Testing evidence must be retained.

16.41 Section Summary

The TODP Testing Architecture establishes a comprehensive quality framework covering functionality, performance, security, governance, and operational reliability.

Implementation

↓

Testing

↓

Validation

↓

Release



Monitoring

The architecture ensures that all platform capabilities remain reliable, secure, auditable, traceable, reproducible, and aligned with TODP's trust-centered principles.

IAD Progress

- 🔗 Section 1 — Repository Structure & Implementation Foundation
- 🔗 Section 2 — Frontend Implementation Architecture (React)
- 🔗 Section 3 — Backend Implementation Architecture (FastAPI)
- 🔗 Section 4 — Database Schema Architecture & PostgreSQL Implementation
- 🔗 Section 5 — Retrieval Implementation Architecture (BM25 + pgvector + Hybrid Search)
- 🔗 Section 6 — Ranking Engine Implementation Architecture
- 🔗 Section 7 — Recommendation Engine Implementation Architecture
- 🔗 Section 8 — Authority Verification Implementation Architecture
- 🔗 Section 9 — Evidence Validation Implementation Architecture
- 🔗 Section 10 — Auditability, Traceability & Reproducibility Implementation Architecture
- 🔗 Section 11 — Event-Driven Architecture & Inter-Service Communication Implementation
- 🔗 Section 12 — Authentication & Authorization Implementation Architecture
- 🔗 Section 13 — API Gateway & External Interface Implementation Architecture
- 🔗 Section 14 — Redis Caching Architecture & Performance Optimization Implementation
- 🔗 Section 15 — Observability Implementation Architecture
- 🔗 Section 16 — Testing & Quality Assurance Implementation Architecture

❏ Section 17 — CI/CD & Deployment Implementation Architecture

IAD V1 — Section 17

CI/CD & Deployment Implementation Architecture

Document: Implementation Architecture Document (IAD V1)

Section: 17

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)

17.1 Purpose

This section defines the Continuous Integration, Continuous Delivery, and Deployment architecture for TODP.

The CI/CD architecture is responsible for:

- Automated builds
- Automated testing

- Automated validation
- Artifact creation
- Environment deployment
- Release governance

The objective is to deliver changes safely, consistently, and reproducibly.

17.2 CI/CD Philosophy

Code changes must move through a governed pipeline.

Developer Change

↓

Validation

↓

Build

↓

Testing

↓

Approval

↓

Deployment

↓

Monitoring

Manual deployments are discouraged except for emergency recovery scenarios.

17.3 CI/CD Principles

CICD-01

Everything is version controlled.

CICD-02

Builds are automated.

CICD-03

Tests are automated.

CICD-04

Deployments are reproducible.

CICD-05

Deployments are auditable.

CICD-06

Production releases are governed.

17.4 CI/CD Architecture

Developer

↓

Git Repository

↓

CI Pipeline



Build Artifacts



Deployment Pipeline



Environment



Monitoring

17.5 Pipeline Objectives

Validate Code

Validate Quality

Validate Security

Validate Governance

Create Artifacts

Deploy Safely

17.6 Source Control Strategy

Repository:

Monorepo

Branches:

main

develop

feature/*

hotfix/*

release/*

17.7 Development Workflow

Feature Branch



Pull Request



Code Review



Automated Validation



Merge



Deployment Pipeline

17.8 Pull Request Requirements

Required Checks:

Unit Tests

Integration Tests

Security Scans

Linting

Code Review

Build Validation

17.9 CI Pipeline Stages

Stage 1

Source Checkout

Stage 2

Dependency Installation

Stage 3

Code Quality Checks

Stage 4

Unit Tests

Stage 5

Integration Tests

Stage 6

Security Scans

Stage 7

Artifact Build

Stage 8

Publish Artifacts

17.10 Code Quality Validation

Backend:

Black

Ruff

MyPy

Frontend:

ESLint

TypeScript Checks

Prettier

17.11 Test Execution

Executed Automatically:

Unit Tests

Integration Tests

Contract Tests

Governance Tests

Security Tests

17.12 Coverage Requirements

Backend Coverage:

≥ 85%

Domain Logic:

≥ 90%

Critical Business Rules:

≥ 95%

Frontend Coverage:

≥ 80%

17.13 Security Validation

Checks:

Dependency Vulnerabilities

Container Vulnerabilities

Secret Detection

Static Analysis

Permission Validation

17.14 Artifact Generation

Artifacts:

Frontend Bundle

Backend Container

Migration Package

Infrastructure Package

Documentation Package

17.15 Container Build Process

Source Code

↓

Docker Build



Container Scan



Version Tag



Artifact Registry

17.16 Versioning Strategy

Format:

MAJOR.MINOR.PATCH

Example:

1.0.0

1.1.0

1.1.1

17.17 Artifact Repository

Stores:

Container Images

Build Outputs

Release Packages

Infrastructure Packages

Deployment Metadata

17.18 Deployment Pipeline

Artifact

↓

Environment Validation



Database Migration



Application Deployment



Health Validation



Monitoring Validation



Release Complete

17.19 Environment Promotion

Development

↓

Test

↓

Stage

↓

Production

Promotion requires successful validation.

17.20 Deployment Strategy

Primary Strategy:

Rolling Deployment

Benefits:

Minimal Downtime

Safer Releases

Controlled Rollout

17.21 Future Deployment Strategies

Blue-Green Deployment

Canary Deployment

Progressive Delivery

Feature Flag Releases

17.22 Database Migration Process

Migration Package

↓

Validation

↓

Execution



Verification



Audit Record

17.23 Rollback Strategy

Rollback Triggers:

Deployment Failure

Critical Defect

Security Issue

Governance Issue

Rollback Targets:

Application

Database

Infrastructure

Configuration

17.24 Health Checks

Backend Health

Database Health

Redis Health

Observability Health

API Health

17.25 Post-Deployment Validation

Validate:

Application Availability

API Availability

Database Connectivity

Cache Connectivity

Telemetry Flow

Security Controls

17.26 Release Governance

Production Release Requires:

Successful Pipeline

Required Coverage

Security Approval

Governance Validation

Deployment Approval

17.27 Deployment Auditability

Every deployment generates:

Deployment Event

↓

Audit Record

↓

Trace Record

↓

Release Record

17.28 Release Metadata

Stored Information:

Release Version

Commit Hash

Build Number

Deployment Time

Approver

Environment

17.29 CI/CD Observability

Metrics:

Build Duration

Build Success Rate

Deployment Duration

Deployment Success Rate

Rollback Count

Release Frequency

17.30 Failure Handling

Failure Types:

Build Failure

Test Failure

Security Failure

Deployment Failure

Migration Failure

Validation Failure

All failures must be logged and audited.

17.31 Deployment Performance Targets

Build Pipeline:

< 20 minutes

Container Build:

< 10 minutes

Application Deployment:

< 15 minutes

Rollback:

< 10 minutes

17.32 CI/CD Repository Structure

.github/

```
|— workflows/
| |— build.yml
| |— test.yml
| |— security.yml
| |— deploy.yml
| |— release.yml
```

17.33 Testing Requirements

Validate:

Build Pipeline

Deployment Pipeline

Rollback Process

Migration Process

Environment Promotion

Release Governance

17.34 Governance Rules

CICD-GOV-01

All deployments use approved pipelines.

CICD-GOV-02

Production releases require validation.

CICD-GOV-03

Production releases require audit records.

CICD-GOV-04

Rollback procedures must exist.

CICD-GOV-05

Release history must be preserved.

CICD-GOV-06

Deployment metadata must be traceable.

17.35 Architectural Constraints

AC-CICD-01

Production deployments bypassing CI/CD are prohibited.

AC-CICD-02

Failed tests block release.

AC-CICD-03

Security failures block release.

AC-CICD-04

Governance failures block release.

AC-CICD-05

Release history remains preserved.

AC-CICD-06

Deployment actions remain auditable.

17.36 Section Summary

The TODP CI/CD Architecture provides a governed, automated, and reproducible framework for software delivery.

Code Change



Validation



Build



Testing



Artifact Creation



Deployment



Monitoring



Governance

The architecture ensures safe releases while preserving security, auditability, traceability, reproducibility, and operational reliability.

IAD Progress

- 📄 Section 1 — Repository Structure & Implementation Foundation
- 📄 Section 2 — Frontend Implementation Architecture (React)
- 📄 Section 3 — Backend Implementation Architecture (FastAPI)
- 📄 Section 4 — Database Schema Architecture & PostgreSQL Implementation
- 📄 Section 5 — Retrieval Implementation Architecture (BM25 + pgvector + Hybrid Search)
- 📄 Section 6 — Ranking Engine Implementation Architecture
- 📄 Section 7 — Recommendation Engine Implementation Architecture
- 📄 Section 8 — Authority Verification Implementation Architecture
- 📄 Section 9 — Evidence Validation Implementation Architecture
- 📄 Section 10 — Auditability, Traceability & Reproducibility Implementation Architecture
- 📄 Section 11 — Event-Driven Architecture & Inter-Service Communication Implementation
- 📄 Section 12 — Authentication & Authorization Implementation Architecture
- 📄 Section 13 — API Gateway & External Interface Implementation Architecture
- 📄 Section 14 — Redis Caching Architecture & Performance Optimization Implementation
- 📄 Section 15 — Observability Implementation Architecture
- 📄 Section 16 — Testing & Quality Assurance Implementation Architecture
- 📄 Section 17 — CI/CD & Deployment Implementation Architecture
- 📄 Section 18 — Infrastructure as Code & Environment Provisioning Architecture

IAD V1 — Section 18

Infrastructure as Code (IaC) & Environment Provisioning Architecture

Document: Implementation Architecture Document (IAD V1)

Section: 18

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)

18.1 Purpose

This section defines the Infrastructure as Code (IaC) and Environment Provisioning architecture for TODP.

The infrastructure architecture is responsible for:

- Environment provisioning
- Infrastructure consistency
- Infrastructure versioning
- Environment reproducibility

- Deployment readiness
- Operational scalability

All infrastructure must be provisioned through code.

Manual infrastructure creation is prohibited.

18.2 Infrastructure Philosophy

Infrastructure is treated as software.

Infrastructure



Version Controlled



Reviewed



Tested



Provisioned



Audited

18.3 Infrastructure Principles

IAC-01

Infrastructure is code.

IAC-02

Infrastructure is versioned.

IAC-03

Infrastructure is reproducible.

IAC-04

Infrastructure changes are auditable.

IAC-05

Infrastructure changes are traceable.

IAC-06

Manual production changes are prohibited.

18.4 Environment Architecture

Development

↓

Test

↓

Stage

↓

Production

Each environment is independently provisioned.

18.5 Environment Objectives

Development

Developer productivity

Test

Automated validation

Stage

Production simulation

Production

Business operations

18.6 Infrastructure Components

Component 1

Network Layer

Component 2

Application Layer

Component 3

Database Layer

Component 4

Cache Layer

Component 5

Observability Layer

Component 6

Security Layer

18.7 Infrastructure Repository Structure

infrastructure/

├─ environments/

| └─ dev/

| └─ test/

| └─ stage/

| └─ prod/

|

├─ networking/

├─ compute/

├─ database/

├─ cache/

├─ observability/

├─ security/

└─ modules/

18.8 Environment Configuration

Each environment defines:

Environment Variables

Resource Sizes

Network Settings

Security Policies

Monitoring Configuration

Backup Policies

18.9 Compute Layer

Hosts:

Frontend

Backend

Background Workers

Observability Services

18.10 Container Architecture

All services are containerized.

Containers:

Frontend

Backend

Redis

Prometheus

Grafana

Telemetry Collectors

18.11 Container Standards

Requirements:

Immutable Images

Versioned Images

Security Scanning

Health Checks

Resource Limits

18.12 Network Architecture

Public Zone



API Layer



Application Layer



Data Layer

Network access is restricted by role.

18.13 Database Provisioning

Provision:

PostgreSQL

pgvector Extension

Database Users

Database Roles

Backup Policies

Monitoring Policies

18.14 Cache Provisioning

Provision:

Redis

Memory Limits

Persistence Settings

Monitoring Configuration

Security Controls

18.15 Observability Provisioning

Provision:

OpenTelemetry

Prometheus

Grafana

Alert Rules

Dashboards

18.16 Security Provisioning

Provision:

Secrets Storage

TLS Certificates

Network Policies

Access Controls

Audit Logging

18.17 Secrets Management

Managed Secrets:

Database Credentials

JWT Secrets

API Keys

Encryption Keys

Service Credentials

18.18 Secrets Rules

Secrets must:

Never exist in source code

Never exist in Git repositories

Never exist in container images

Be rotated periodically

18.19 Environment Variables

Examples:

DATABASE_URL

REDIS_URL

JWT_SECRET

ENVIRONMENT

LOG_LEVEL

18.20 Configuration Management

Configuration Categories:

Application Configuration

Infrastructure Configuration

Security Configuration

Observability Configuration

Governance Configuration

18.21 Infrastructure Versioning

Every infrastructure change requires:

Version Control

Review

Approval

Deployment Record

Audit Record

18.22 Infrastructure Deployment Workflow

Infrastructure Change

↓

Pull Request

↓

Review

↓

Approval



Provisioning



Validation



Monitoring

18.23 Infrastructure Validation

Validate:

Network Connectivity

Application Availability

Database Connectivity

Redis Connectivity

Monitoring Connectivity

Security Controls

18.24 Environment Reproducibility

Requirement:

Entire environments can be recreated from source control.

Environment Recreation Includes:

Infrastructure

Configuration

Monitoring

Security Controls

18.25 Backup Provisioning

Provision:

Database Backups

Configuration Backups

Infrastructure State Backups

Observability Backups

18.26 Disaster Recovery Readiness

Infrastructure must support:

Environment Recovery

Database Recovery

Application Recovery

Observability Recovery

Security Recovery

18.27 Infrastructure Observability

Metrics:

Provision Success Rate

Deployment Success Rate

Infrastructure Failures

Resource Utilization

Environment Health

18.28 Infrastructure Auditability

All infrastructure actions generate:

Infrastructure Event

↓

Audit Record



Trace Record



Deployment Record

18.29 Infrastructure Testing

Validate:

Provisioning Scripts

Environment Builds

Network Policies

Security Policies

Monitoring Configuration

Backup Configuration

18.30 Infrastructure Performance Requirements

Environment Provisioning:

< 30 minutes

Container Startup:

< 60 seconds

Database Provisioning:

< 15 minutes

Cache Provisioning:

< 5 minutes

18.31 Environment Promotion Model

Development

↓

Test

↓

Stage



Production

Promotion requires validation at each stage.

18.32 Governance Rules

IAC-GOV-01

Infrastructure must be code-defined.

IAC-GOV-02

Infrastructure changes require review.

IAC-GOV-03

Infrastructure changes require audit records.

IAC-GOV-04

Production changes require approval.

IAC-GOV-05

Environment recreation must be possible.

IAC-GOV-06

Infrastructure history must be preserved.

18.33 Architectural Constraints

AC-IAC-01

Manual production provisioning is prohibited.

AC-IAC-02

Infrastructure must remain reproducible.

AC-IAC-03

Infrastructure changes must be versioned.

AC-IAC-04

Secrets cannot be stored in source code.

AC-IAC-05

Infrastructure changes must be auditable.

AC-IAC-06

Governance controls cannot be bypassed.

18.34 Section Summary

The TODP Infrastructure as Code Architecture establishes a reproducible, auditable, and governed framework for provisioning and managing all platform environments.

Infrastructure Code



Provisioning



Validation



Deployment



Monitoring



Governance

The architecture ensures consistency, scalability, security, auditability, traceability, and operational reliability across all TODP environments.

IAD Progress

☑ Sections 1–16 Complete

☑ Section 17 — CI/CD & Deployment Implementation Architecture

☑ Section 18 — Infrastructure as Code & Environment Provisioning Architecture

☑ Section 19 — Security Implementation Architecture

IAD V1 — Section 19

Security Implementation Architecture

Document: Implementation Architecture Document (IAD V1)

Section: 19

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)

19.1 Purpose

This section defines the security implementation architecture for TODP.

The security architecture is responsible for:

- Confidentiality
- Integrity
- Availability
- Identity Protection
- Access Control
- Data Protection
- Operational Security
- Governance Compliance

Security is implemented as a platform-wide capability.

Every component participates in security.

19.2 Security Philosophy

Security is not a feature.

Security is an architectural property.

Security must be:

- Preventive
- Detective
- Corrective
- Auditable
- Traceable
- Governed

19.3 Security Principles

SEC-01

Least Privilege

SEC-02

Defense in Depth

SEC-03

Zero Trust Principles

SEC-04

Secure by Default

SEC-05

Security Observability

SEC-06

Security Auditability

19.4 Security Architecture

User



Authentication



Authorization



API Security



Application Security



Infrastructure Security



Data Security



Audit & Monitoring

19.5 Security Domains

Domain 1

Identity Security

Domain 2

Application Security

Domain 3

API Security

Domain 4

Infrastructure Security

Domain 5

Database Security

Domain 6

Operational Security

19.6 Identity Security

Responsible for:

Authentication

Authorization

Session Management

Role Management

Permission Enforcement

19.7 User Authentication

Technology:

JWT

Access Tokens

Refresh Tokens

Role Resolution

Permission Resolution

19.8 Password Security

Requirements:

Strong Passwords

Secure Hashing

Password Reset

Password Rotation

Account Lockout

Implementation:

bcrypt

or equivalent secure algorithm

19.9 Session Security

Controls:

Session Tracking

Token Expiration

Token Revocation

Session Timeout

Concurrent Session Monitoring

19.10 Authorization Security

Model:

RBAC

Role-Based Access Control

Roles:

User

Reviewer

Administrator

Governance Administrator

Security Administrator

System Administrator

19.11 Permission Security

Permissions Example:

opportunity.read

authority.verify

evidence.validate

ranking.execute

recommendation.generate

audit.read

policy.manage

19.12 API Security

Controls:

JWT Validation

Permission Validation

Input Validation

Rate Limiting

Security Logging

Request Auditing

19.13 Input Validation

All external inputs must be validated.

Validation Types:

Schema Validation

Type Validation

Length Validation

Format Validation

Business Rule Validation

19.14 Output Security

Responses must:

Exclude secrets

Exclude internal errors

Exclude credentials

Exclude sensitive metadata

19.15 API Rate Limiting

Purpose:

Abuse Prevention

Resource Protection

Security Protection

Anonymous:

100 Requests/Hour

Authenticated:

1000 Requests/Hour

19.16 Secure Headers

Required Headers:

HSTS

X-Frame-Options

X-Content-Type-Options

Referrer-Policy

Content-Security-Policy

19.17 Infrastructure Security

Protects:

Servers

Containers

Networks

Storage

Monitoring Systems

Deployment Systems

19.18 Network Security

Architecture:

Public Zone

↓

API Layer

↓

Application Layer

↓

Data Layer

Access restricted by policy.

19.19 Container Security

Requirements:

Minimal Images

Immutable Images

Security Scanning

Signed Images

Resource Limits

19.20 Database Security

Controls:

Role-Based Access

Encryption

Audit Logging

Connection Security

Backup Protection

19.21 Database Access Rules

Applications use:

Dedicated Service Accounts

Users never access database directly.

Production database access is restricted.

19.22 Encryption Standards

Encryption In Transit:

TLS

Encryption At Rest:

Database Encryption

Backup Encryption

Storage Encryption

19.23 Secrets Management

Managed Secrets:

Database Credentials

JWT Secrets

API Keys

Encryption Keys

Service Credentials

19.24 Secrets Rules

Secrets:

Never stored in Git

Never stored in source code

Never exposed through APIs

Must be rotated periodically

19.25 Data Security

Protect:

User Data

Authority Data

Evidence Data

Governance Data

Audit Data

Trace Data

19.26 Audit Security

Security Events Logged:

Login

Logout

Failed Login

Permission Denied

Role Change

Policy Change

Configuration Change

19.27 Security Monitoring

Monitored Events:

Authentication Events

Authorization Events

API Events

Database Events

Infrastructure Events

Governance Events

19.28 Threat Detection

Detect:

Brute Force Attempts

Repeated Failures

Unauthorized Access

Privilege Escalation

Abnormal Activity

19.29 Vulnerability Management

Activities:

Scanning

Assessment

Prioritization

Remediation

Verification

19.30 Dependency Security

Validate:

Python Dependencies

Node Dependencies

Container Dependencies

Infrastructure Dependencies

19.31 Logging Requirements

Every security event includes:

Timestamp

User

Action

Result

Correlation ID

Source

19.32 Security Incident Response

Process:

Detection



Investigation



Containment



Remediation



Validation



Closure

19.33 Security Testing

Validate:

Authentication

Authorization

Input Validation

Session Security

API Security

Infrastructure Security

19.34 Penetration Testing

Scope:

API Layer

Application Layer

Authentication

Authorization

Infrastructure

Frequency:

Before Production Releases

Periodic Reviews

19.35 Security Observability

Metrics:

Failed Logins

Authorization Failures

Security Alerts

Rate Limit Violations

Token Failures

Suspicious Activity

19.36 Security Auditability

Every security action generates:

Security Event



Audit Record



Trace Record



Correlation ID

19.37 Security Recovery

Recovery Capabilities:

Credential Rotation

Session Revocation

Access Revocation

Service Isolation

Emergency Lockdown

19.38 Governance Rules

SEC-GOV-01

All users must authenticate.

SEC-GOV-02

All actions must authorize.

SEC-GOV-03

Security events must be audited.

SEC-GOV-04

Secrets must be protected.

SEC-GOV-05

Security incidents must be traceable.

SEC-GOV-06

Security controls require validation.

19.39 Architectural Constraints

AC-SEC-01

Authentication cannot be bypassed.

AC-SEC-02

Authorization cannot be bypassed.

AC-SEC-03

Secrets cannot exist in source code.

AC-SEC-04

Production database access is restricted.

AC-SEC-05

Security events remain auditable.

AC-SEC-06

Governance controls cannot bypass security.

19.40 Section Summary

The TODP Security Architecture establishes a defense-in-depth framework protecting users, services, infrastructure, and data.

Identity



Authorization



Application Security



Infrastructure Security



Data Protection



Monitoring



Auditability

The architecture ensures confidentiality, integrity, availability, traceability, auditability, and governance compliance throughout the platform.

IAD Progress

- 📄 Section 1 — Repository Structure & Implementation Foundation
- 📄 Section 2 — Frontend Implementation Architecture (React)
- 📄 Section 3 — Backend Implementation Architecture (FastAPI)
- 📄 Section 4 — Database Schema Architecture & PostgreSQL Implementation
- 📄 Section 5 — Retrieval Implementation Architecture (BM25 + pgvector + Hybrid Search)
- 📄 Section 6 — Ranking Engine Implementation Architecture
- 📄 Section 7 — Recommendation Engine Implementation Architecture
- 📄 Section 8 — Authority Verification Implementation Architecture
- 📄 Section 9 — Evidence Validation Implementation Architecture
- 📄 Section 10 — Auditability, Traceability & Reproducibility Implementation Architecture
- 📄 Section 11 — Event-Driven Architecture & Inter-Service Communication Implementation
- 📄 Section 12 — Authentication & Authorization Implementation Architecture
- 📄 Section 13 — API Gateway & External Interface Implementation Architecture
- 📄 Section 14 — Redis Caching Architecture & Performance Optimization Implementation
- 📄 Section 15 — Observability Implementation Architecture
- 📄 Section 16 — Testing & Quality Assurance Implementation Architecture
- 📄 Section 17 — CI/CD & Deployment Implementation Architecture
- 📄 Section 18 — Infrastructure as Code & Environment Provisioning Architecture
- 📄 Section 19 — Security Implementation Architecture

❏ Section 20 — Disaster Recovery & Backup Implementation Architecture

IAD V1 — Section 20

Disaster Recovery & Backup Implementation Architecture

Document: Implementation Architecture Document (IAD V1)

Section: 20

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)

20.1 Purpose

This section defines the Disaster Recovery (DR) and Backup implementation architecture for TODP.

The DR architecture is responsible for:

- Business continuity
- Data protection
- Service recovery

- Environment recovery
- Incident recovery
- Operational resilience

The objective is to ensure TODP can recover from failures while preserving trust, traceability, auditability, and reproducibility.

20.2 Recovery Philosophy

Failures are inevitable.

Data loss is unacceptable.

Recovery must be:

- Planned
- Tested
- Repeatable
- Auditable
- Governed

Recovery procedures must exist before production launch.

20.3 Recovery Principles

DR-01

Critical data must be backed up.

DR-02

Recovery procedures must be documented.

DR-03

Recovery procedures must be tested.

DR-04

Recovery activities must be auditable.

DR-05

Recovery objectives must be measurable.

DR-06

Recovery processes must be reproducible.

20.4 Recovery Architecture

Production Environment



Backup Systems



Recovery Procedures



Validation



Operational Restoration

20.5 Disaster Categories

Category 1

Application Failure

Category 2

Database Failure

Category 3

Infrastructure Failure

Category 4

Network Failure

Category 5

Security Incident

Category 6

Human Error

20.6 Recovery Objectives

Recovery Time Objective (RTO)

Maximum acceptable downtime.

Recovery Point Objective (RPO)

Maximum acceptable data loss.

20.7 TODP Recovery Targets

Application Services

RTO < 60 Minutes

RPO < 15 Minutes

Database

RTO < 60 Minutes

RPO < 15 Minutes

Observability Stack

RTO < 120 Minutes

RPO < 60 Minutes

20.8 Backup Categories

Operational Backups

Database Backups

Configuration Backups

Infrastructure Backups

Observability Backups

Security Backups

20.9 Database Backup Strategy

Protected Assets:

PostgreSQL

pgvector Data

Indexes

Schemas

Roles

Permissions

20.10 Database Backup Schedule

Incremental Backup

Every Hour

Full Backup

Daily

Long-Term Archive

Weekly

20.11 Backup Retention Policy

Hourly Backups

7 Days

Daily Backups

30 Days

Weekly Backups

12 Months

Governance Archives

As Required By Policy

20.12 Backup Storage

Primary Backup Storage

↓

Secondary Backup Storage

↓

Archive Storage

Multiple copies maintained.

20.13 Configuration Backup

Protect:

Environment Variables

Infrastructure Configuration

Application Configuration

Governance Configuration

Monitoring Configuration

20.14 Infrastructure Backup

Protect:

Infrastructure Definitions

Deployment Configurations

Container Definitions

Network Policies

Security Policies

20.15 Security Backup

Protect:

Certificates

Encryption Configurations

Security Policies

Access Configurations

Audit Configurations

20.16 Observability Backup

Protect:

Dashboards

Alert Rules

Metric Definitions

Telemetry Configuration

Monitoring Policies

20.17 Recovery Repository Structure

docs/RUNBOOKS/

- |— database_recovery.md
- |— application_recovery.md
- |— infrastructure_recovery.md
- |— security_recovery.md
- |— observability_recovery.md
- └─ disaster_recovery.md

20.18 Application Recovery

Recovery Process:

Identify Failure



Restore Application



Validate Services



Validate APIs



Resume Operations

20.19 Database Recovery

Recovery Process:

Identify Failure



Restore Backup



Replay Logs



Validate Data



Resume Operations

20.20 Infrastructure Recovery

Recovery Process:

Load IaC



Provision Environment



Deploy Services



Validate Environment



Resume Operations

20.21 Security Incident Recovery

Examples:

Credential Compromise

Token Leakage

Unauthorized Access

Privilege Escalation

Recovery Actions:

Rotate Secrets

Revoke Access

Audit Investigation

Validation

20.22 Service Dependency Recovery

Recovery Priority:

1. Network

2. Database

3. Cache

4. Backend Services

5. Frontend Services

6. Observability

20.23 Recovery Validation

Validate:

Application Availability

Database Availability

Search Availability

Ranking Availability

Recommendation Availability

Governance Availability

20.24 Backup Validation

Every backup must be verified.

Validation Includes:

Backup Integrity

Recovery Testing

Data Verification

Checksum Verification

20.25 Recovery Testing

Required Tests:

Backup Restore

Database Recovery

Application Recovery

Infrastructure Recovery

Security Recovery

Frequency:

Quarterly

Or before major releases.

20.26 Business Continuity

Critical Services:

Authentication

Opportunity Discovery

Authority Verification

Evidence Validation

Ranking

Recommendation

Audit Services

20.27 Recovery Automation

Automate:

Backup Execution

Backup Verification

Recovery Validation

Health Verification

Notification Workflows

20.28 Recovery Monitoring

Metrics:

Backup Success Rate

Backup Failures

Recovery Duration

Recovery Success Rate

RPO Compliance

RTO Compliance

20.29 Alerting

Generate Alerts For:

Backup Failure

Recovery Failure

Restore Failure

Storage Failure

Replication Failure

20.30 Auditability

Every recovery action generates:

Recovery Event

↓

Audit Record

↓

Trace Record

↓

Recovery Report

20.31 Recovery Reports

Include:

Incident Summary

Recovery Timeline

Affected Systems

Recovery Actions

Validation Results

Lessons Learned

20.32 Recovery Testing Requirements

Unit Tests:

Backup Utilities

Recovery Utilities

Validation Utilities

Integration Tests:

Database Recovery

Infrastructure Recovery

Application Recovery

Disaster Simulations:

Environment Failure

Database Failure

Security Incident

20.33 Governance Rules

DR-GOV-01

Critical data requires backup.

DR-GOV-02

Recovery procedures must be documented.

DR-GOV-03

Recovery procedures must be tested.

DR-GOV-04

Recovery actions must be auditable.

DR-GOV-05

Backup integrity must be validated.

DR-GOV-06

Recovery metrics must be monitored.

20.34 Architectural Constraints

AC-DR-01

Production systems require backup coverage.

AC-DR-02

Recovery procedures must exist before launch.

AC-DR-03

Recovery testing is mandatory.

AC-DR-04

Recovery actions remain auditable.

AC-DR-05

Critical systems require defined RTO/RPO.

AC-DR-06

Governance controls cannot bypass recovery requirements.

20.35 Section Summary

The TODP Disaster Recovery Architecture establishes a resilient framework for protecting data, restoring services, and maintaining operational continuity.

Backup



Validation



Recovery



Verification



Operational Restoration



Governance Review

The architecture ensures resilience, recoverability, auditability, traceability, and long-term operational trust.

IAD Progress

- 🔗 Section 1 — Repository Structure & Implementation Foundation
- 🔗 Section 2 — Frontend Implementation Architecture (React)
- 🔗 Section 3 — Backend Implementation Architecture (FastAPI)
- 🔗 Section 4 — Database Schema Architecture & PostgreSQL Implementation
- 🔗 Section 5 — Retrieval Implementation Architecture (BM25 + pgvector + Hybrid Search)
- 🔗 Section 6 — Ranking Engine Implementation Architecture
- 🔗 Section 7 — Recommendation Engine Implementation Architecture
- 🔗 Section 8 — Authority Verification Implementation Architecture
- 🔗 Section 9 — Evidence Validation Implementation Architecture
- 🔗 Section 10 — Auditability, Traceability & Reproducibility Implementation Architecture
- 🔗 Section 11 — Event-Driven Architecture & Inter-Service Communication Implementation
- 🔗 Section 12 — Authentication & Authorization Implementation Architecture
- 🔗 Section 13 — API Gateway & External Interface Implementation Architecture
- 🔗 Section 14 — Redis Caching Architecture & Performance Optimization Implementation
- 🔗 Section 15 — Observability Implementation Architecture
- 🔗 Section 16 — Testing & Quality Assurance Implementation Architecture
- 🔗 Section 17 — CI/CD & Deployment Implementation Architecture
- 🔗 Section 18 — Infrastructure as Code & Environment Provisioning Architecture
- 🔗 Section 19 — Security Implementation Architecture
- 🔗 Section 20 — Disaster Recovery & Backup Implementation Architecture

🔗 Section 21 — Operational Runbooks & Platform Administration Architecture

IAD V1 — Section 21

Operational Runbooks & Platform Administration Architecture

Document: Implementation Architecture Document (IAD V1)

Section: 21

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)

21.1 Purpose

This section defines the Operational Runbooks and Platform Administration architecture for TODP.

The operational architecture is responsible for:

- Day-to-day platform operations
- Incident response
- Service administration
- Environment management
- Governance operations
- Operational continuity

Operational procedures ensure that TODP remains reliable, secure, auditable, and maintainable.

21.2 Operational Philosophy

Systems are not self-managing.

Operational excellence requires:

- Documented procedures
- Defined ownership
- Repeatable processes
- Auditable actions
- Continuous monitoring

Every critical operational activity must have a runbook.

21.3 Operational Principles

OPS-01

Critical procedures must be documented.

OPS-02

Operational actions must be auditable.

OPS-03

Operational actions must be traceable.

OPS-04

Operational procedures must be repeatable.

OPS-05

Operational ownership must be defined.

OPS-06

Operational readiness must be measurable.

21.4 Operational Architecture

Monitoring

↓

Alert

↓

Runbook

↓

Investigation

↓

Resolution

↓

Validation

↓

Audit

21.5 Runbook Repository Structure

docs/RUNBOOKS/

└─ incident_response.md

- |— deployment_operations.md
- |— database_operations.md
- |— cache_operations.md
- |— security_operations.md
- |— observability_operations.md
- |— governance_operations.md
- |— backup_recovery.md
- └─ platform_administration.md

21.6 Operational Domains

Domain 1

Application Operations

Domain 2

Database Operations

Domain 3

Infrastructure Operations

Domain 4

Security Operations

Domain 5

Governance Operations

Domain 6

Observability Operations

21.7 Platform Administration

Responsible for:

Platform Configuration

System Settings

Feature Management

Environment Management

Operational Governance

21.8 Administrative Roles

System Administrator

Security Administrator

Governance Administrator

Operations Administrator

Reviewer

User

21.9 Administrative Responsibilities

System Administrator

Infrastructure and platform operations.

Security Administrator

Security controls and access management.

Governance Administrator

Policy and governance operations.

Operations Administrator

Monitoring and incident management.

21.10 Service Inventory

Managed Services:

Frontend

Backend

PostgreSQL

Redis

Prometheus

Grafana

OpenTelemetry

Background Workers

21.11 Service Health Management

Health Categories:

Healthy

Degraded

Unavailable

Maintenance

21.12 Operational Dashboard

Displays:

System Status

Service Health

Resource Utilization

Open Incidents

Active Alerts

Recovery Status

21.13 Incident Management

Incident Lifecycle:

Detection

↓

Classification

↓

Investigation



Mitigation



Resolution



Review

21.14 Incident Severity Levels

SEV-1

Critical Platform Outage

SEV-2

Major Functionality Impact

SEV-3

Limited Functionality Impact

SEV-4

Minor Issue

21.15 Incident Response Runbook

Step 1

Acknowledge Alert

Step 2

Assess Severity

Step 3

Identify Impact

Step 4

Mitigate Issue

Step 5

Restore Service

Step 6

Validate Recovery

Step 7

Document Incident

21.16 Change Management

Every operational change requires:

Change Request

Risk Assessment

Approval

Execution

Validation

Audit Record

21.17 Maintenance Operations

Examples:

System Updates

Security Updates

Database Maintenance

Infrastructure Updates

Configuration Changes

21.18 Scheduled Maintenance

Requirements:

Planned Window

Approval

Communication

Validation

Audit Record

21.19 Database Operations

Activities:

Backup Validation

Performance Monitoring

Migration Execution

Recovery Validation

Capacity Management

21.20 Cache Operations

Activities:

Cache Monitoring

Memory Monitoring

TTL Validation

Cache Invalidation

Failure Recovery

21.21 Security Operations

Activities:

User Access Reviews

Role Reviews

Secret Rotation

Incident Reviews

Threat Monitoring

21.22 Governance Operations

Activities:

Policy Reviews

Decision Reviews

Audit Reviews

Compliance Reviews

Traceability Reviews

21.23 Observability Operations

Activities:

Dashboard Reviews

Alert Reviews

Metric Validation

Trace Validation

Log Validation

21.24 Capacity Management

Monitor:

CPU

Memory

Storage

Network

Database Growth

Cache Growth

21.25 User Administration

Functions:

User Creation

User Suspension

Role Assignment

Permission Assignment

Access Revocation

21.26 Configuration Management

Managed Configurations:

Application

Infrastructure

Security

Observability

Governance

21.27 Escalation Model

Level 1

Operations Team



Level 2

Platform Owners



Level 3

Executive Governance

21.28 Post-Incident Review

Includes:

Root Cause Analysis

Timeline

Impact Assessment

Corrective Actions

Preventive Actions

21.29 Operational Auditability

All operational actions generate:

Operational Event

↓

Audit Record

↓

Trace Record

↓

Change Record

21.30 Operational Metrics

Metrics:

Incident Count

Incident Duration

MTTR

Availability

Change Success Rate

Operational Errors

21.31 Availability Targets

Core Platform Availability:

99.9%

Critical Services Availability:

99.95%

21.32 Operational Reporting

Reports:

Weekly Operations Report

Monthly Reliability Report

Security Operations Report

Governance Operations Report

21.33 Operational Testing

Validate:

Runbooks

Escalation Procedures

Recovery Procedures

Change Procedures

Incident Procedures

21.34 Documentation Standards

Runbooks must include:

Purpose

Prerequisites

Procedure

Validation

Rollback

Escalation

Ownership

21.35 Governance Rules

OPS-GOV-01

Critical procedures require runbooks.

OPS-GOV-02

Operational changes require approval.

OPS-GOV-03

Operational actions require audit records.

OPS-GOV-04

Incidents require review.

OPS-GOV-05

Operational ownership must be defined.

OPS-GOV-06

Runbooks must remain current.

21.36 Architectural Constraints

AC-OPS-01

Critical operations require documented procedures.

AC-OPS-02

Operational actions remain auditable.

AC-OPS-03

Operational actions remain traceable.

AC-OPS-04

Unauthorized administrative actions are prohibited.

AC-OPS-05

Runbooks require periodic review.

AC-OPS-06

Governance controls cannot bypass operational procedures.

21.37 Section Summary

The TODP Operational Runbook Architecture establishes a governed framework for operating, maintaining, and supporting the platform.

Monitoring



Alerting



Runbook Execution



Resolution



Validation



Audit



Continuous Improvement

The architecture ensures operational consistency, reliability, security, auditability, traceability, and long-term maintainability.

IAD Progress

- 📄 Section 1 — Repository Structure & Implementation Foundation
- 📄 Section 2 — Frontend Implementation Architecture (React)
- 📄 Section 3 — Backend Implementation Architecture (FastAPI)
- 📄 Section 4 — Database Schema Architecture & PostgreSQL Implementation
- 📄 Section 5 — Retrieval Implementation Architecture (BM25 + pgvector + Hybrid Search)
- 📄 Section 6 — Ranking Engine Implementation Architecture
- 📄 Section 7 — Recommendation Engine Implementation Architecture
- 📄 Section 8 — Authority Verification Implementation Architecture
- 📄 Section 9 — Evidence Validation Implementation Architecture
- 📄 Section 10 — Auditability, Traceability & Reproducibility Implementation Architecture
- 📄 Section 11 — Event-Driven Architecture & Inter-Service Communication Implementation
- 📄 Section 12 — Authentication & Authorization Implementation Architecture
- 📄 Section 13 — API Gateway & External Interface Implementation Architecture
- 📄 Section 14 — Redis Caching Architecture & Performance Optimization Implementation
- 📄 Section 15 — Observability Implementation Architecture
- 📄 Section 16 — Testing & Quality Assurance Implementation Architecture

☐ Section 17 — CI/CD & Deployment Implementation Architecture

☐ Section 18 — Infrastructure as Code & Environment Provisioning Architecture

☐ Section 19 — Security Implementation Architecture

☐ Section 20 — Disaster Recovery & Backup Implementation Architecture

☐ Section 21 — Operational Runbooks & Platform Administration Architecture

☐ Section 22 — Development Standards & Coding Guidelines Architecture

IAD V1 — Section 22

Development Standards & Coding Guidelines Architecture

Document: Implementation Architecture Document (IAD V1)

Section: 22

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)

- ADD V1 (Frozen)

- DDD V1 (Frozen)

- SDD V1 (Frozen)

22.1 Purpose

This section defines the development standards, coding guidelines, engineering practices, and implementation conventions for TODP.

The objective is to ensure:

- Consistency
- Maintainability
- Scalability
- Readability
- Reliability
- Auditability

All implementation work must follow these standards.

22.2 Engineering Philosophy

Code is a long-term asset.

Code should be:

- Readable
- Testable
- Observable
- Maintainable
- Secure
- Governed

The platform must optimize for long-term sustainability rather than short-term speed.

22.3 Development Principles

DEV-01

Consistency over creativity.

DEV-02

Explicit over implicit.

DEV-03

Simple over complex.

DEV-04

Testable over convenient.

DEV-05

Observable over opaque.

DEV-06

Governed over ad hoc.

22.4 Repository Standards

Repository Type:

Monorepo

Primary Directories:

frontend/

backend/

infrastructure/

docs/

tests/

scripts/

22.5 Branching Standards

Allowed Branch Types:

feature/*

bugfix/*

release/*

hotfix/*

Protected Branches:

main

develop

22.6 Pull Request Standards

Every PR must include:

Purpose

Scope

Testing Evidence

Risk Assessment

Reviewer Approval

22.7 Code Review Requirements

Required Reviews:

Architecture Review

Code Quality Review

Security Review

Testing Review

No self-approval allowed.

22.8 Backend Standards

Language:

Python

Framework:

FastAPI

Style Guide:

PEP 8

Type Hints Mandatory

22.9 Python Standards

Requirements:

Type Hints

Docstrings

Explicit Imports

Dependency Injection

Exception Handling

22.10 Backend Structure Standards

backend/app/

api/

domains/

services/

repositories/

models/

schemas/

security/

governance/

Business logic must not exist in API routes.

22.11 Frontend Standards

Language:

TypeScript

Framework:

React

Architecture:

Component Based

22.12 Frontend Structure

frontend/src/

components/

pages/

hooks/

services/

types/

utils/

contexts/

22.13 Component Standards

Components must:

Be reusable

Be typed

Be testable

Remain focused

Avoid excessive responsibilities

22.14 Naming Standards

Variables:

camelCase

Functions:

camelCase

Classes:

PascalCase

Files:

snake_case (backend)

PascalCase (React components)

22.15 API Standards

RESTful Design

Versioned APIs

Consistent Response Format

OpenAPI Documentation

Error Standardization

22.16 Database Standards

Schema Naming:

snake_case

Primary Keys:

UUID

Foreign Keys:

Mandatory

Indexes:

Explicitly defined

22.17 Migration Standards

Every schema change requires:

Migration

Review

Testing

Rollback Validation

Manual schema changes prohibited.

22.18 Error Handling Standards

Requirements:

Meaningful Messages

Structured Responses

Correlation IDs

Audit Records

Logging

22.19 Logging Standards

All logs must include:

Timestamp

Service

Severity

Correlation ID

Context

Message

22.20 Security Standards

Requirements:

Input Validation

Output Validation

Authentication

Authorization

Secret Protection

Audit Logging

22.21 Testing Standards

Minimum Requirements:

Unit Tests

Integration Tests

Security Tests

Regression Tests

Critical workflows require automated tests.

22.22 Documentation Standards

Required Documentation:

Architecture

API

Runbooks

Deployment

Security

Governance

22.23 Dependency Management

Requirements:

Version Pinning

Security Validation

License Validation

Periodic Review

Unused dependencies prohibited.

22.24 Configuration Standards

Configuration:

Externalized

Versioned

Environment-Specific

Documented

Auditable

22.25 Code Quality Tools

Backend:

Black

Ruff

MyPy

Pytest

Frontend:

ESLint

Prettier

TypeScript

Vitest

22.26 Complexity Standards

Functions:

Single Responsibility

Classes:

Focused Responsibility

Modules:

Clear Ownership

Avoid unnecessary abstractions.

22.27 Architecture Standards

Domain Driven Design

Layered Architecture

Dependency Inversion

Separation of Concerns

Event-Based Communication

22.28 Observability Standards

Every critical workflow must generate:

Metrics

Logs

Traces

Audit Records

22.29 Governance Standards

Critical actions require:

Auditability

Traceability

Reproducibility

Versioning

Documentation

22.30 Commit Standards

Commit Format:

type(scope): description

Examples:

feat(retrieval): add hybrid retrieval engine

fix(ranking): correct tie resolution logic

docs(iad): update section 22

22.31 Release Standards

Release Requirements:

Approved Code

Successful Tests

Security Validation

Governance Validation

Deployment Approval

22.32 Technical Debt Management

Debt Categories:

Code Debt

Architecture Debt

Infrastructure Debt

Documentation Debt

Security Debt

All debt must be tracked.

22.33 Development Metrics

Metrics:

Code Coverage

Build Success Rate

PR Cycle Time

Review Time

Defect Density

Deployment Success Rate

22.34 Developer Onboarding Standards

Required Knowledge:

Architecture

Security

Governance

Testing

Deployment

Operations

22.35 Governance Rules

DEV-GOV-01

All code requires review.

DEV-GOV-02

All schema changes require migrations.

DEV-GOV-03

Critical workflows require tests.

DEV-GOV-04

Security controls require validation.

DEV-GOV-05

Documentation must remain current.

DEV-GOV-06

Development standards are mandatory.

22.36 Architectural Constraints

AC-DEV-01

Direct production database modification prohibited.

AC-DEV-02

Business logic cannot reside in API routes.

AC-DEV-03

Critical workflows require observability.

AC-DEV-04

Critical workflows require auditability.

AC-DEV-05

Security controls cannot be bypassed.

AC-DEV-06

Governance controls cannot be bypassed.

22.37 Section Summary

The TODP Development Standards Architecture establishes a consistent engineering framework for building, maintaining, and evolving the platform.

Requirements



Implementation



Testing



Review



Deployment



Operations



Governance

The architecture ensures maintainability, consistency, security, auditability, traceability, reproducibility, and long-term platform sustainability.

IAD Progress

- 📄 Section 1 — Repository Structure & Implementation Foundation
- 📄 Section 2 — Frontend Implementation Architecture (React)
- 📄 Section 3 — Backend Implementation Architecture (FastAPI)
- 📄 Section 4 — Database Schema Architecture & PostgreSQL Implementation
- 📄 Section 5 — Retrieval Implementation Architecture (BM25 + pgvector + Hybrid Search)
- 📄 Section 6 — Ranking Engine Implementation Architecture
- 📄 Section 7 — Recommendation Engine Implementation Architecture
- 📄 Section 8 — Authority Verification Implementation Architecture
- 📄 Section 9 — Evidence Validation Implementation Architecture
- 📄 Section 10 — Auditability, Traceability & Reproducibility Implementation Architecture
- 📄 Section 11 — Event-Driven Architecture & Inter-Service Communication Implementation
- 📄 Section 12 — Authentication & Authorization Implementation Architecture
- 📄 Section 13 — API Gateway & External Interface Implementation Architecture
- 📄 Section 14 — Redis Caching Architecture & Performance Optimization Implementation
- 📄 Section 15 — Observability Implementation Architecture
- 📄 Section 16 — Testing & Quality Assurance Implementation Architecture
- 📄 Section 17 — CI/CD & Deployment Implementation Architecture
- 📄 Section 18 — Infrastructure as Code & Environment Provisioning Architecture
- 📄 Section 19 — Security Implementation Architecture
- 📄 Section 20 — Disaster Recovery & Backup Implementation Architecture
- 📄 Section 21 — Operational Runbooks & Platform Administration Architecture
- 📄 Section 22 — Development Standards & Coding Guidelines Architecture

- 📄 Section 23 — Production Readiness Checklist & Go-Live Architecture

IAD V1 — Section 23

Production Readiness Checklist & Go-Live Architecture

Document: Implementation Architecture Document (IAD V1)

Section: 23

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)

23.1 Purpose

This section defines the Production Readiness and Go-Live architecture for TODP.

The purpose is to ensure that no environment, service, feature, or release enters production without satisfying defined readiness criteria.

Production readiness is a governance-controlled process.

Deployment success does not imply production readiness.

23.2 Production Readiness Philosophy

A feature is not production ready because:

- It compiles
- It deploys
- It passes unit tests

A feature becomes production ready only when:

- Functional requirements are satisfied
- Security requirements are satisfied
- Governance requirements are satisfied
- Operational requirements are satisfied
- Recovery requirements are satisfied

23.3 Production Readiness Principles

READY-01

Production readiness is measurable.

READY-02

Production readiness is auditable.

READY-03

Production readiness is traceable.

READY-04

Production readiness requires approval.

READY-05

Production readiness requires evidence.

READY-06

Production readiness is reproducible.

23.4 Go-Live Architecture

Development



Testing



Validation



Stage Approval



Production Readiness Review



Go-Live Approval



Production Deployment



Post-Go-Live Monitoring

23.5 Readiness Domains

Domain 1

Application Readiness

Domain 2

Security Readiness

Domain 3

Infrastructure Readiness

Domain 4

Operational Readiness

Domain 5

Governance Readiness

Domain 6

Recovery Readiness

23.6 Application Readiness

Requirements:

Feature Complete

Requirements Satisfied

API Documentation Complete

Error Handling Complete

Observability Complete

23.7 Backend Readiness Checklist

FastAPI Services Operational

Database Connectivity Verified

Redis Connectivity Verified

Authentication Verified

Authorization Verified

Telemetry Verified

23.8 Frontend Readiness Checklist

React Application Verified

API Integration Verified

Error States Verified

Loading States Verified

Accessibility Reviewed

Performance Validated

23.9 Database Readiness Checklist

Schema Validated

Indexes Validated

Migrations Validated

Rollback Validated

Backup Validated

Monitoring Enabled

23.10 Retrieval Readiness Checklist

BM25 Validated

Vector Search Validated

Hybrid Search Validated

Search Performance Validated

Search Observability Enabled

23.11 Ranking Readiness Checklist

Money Cost Calculation Verified

Time Cost Calculation Verified

Distance Cost Calculation Verified

Tie Resolution Verified

Ranking Reproducibility Verified

23.12 Recommendation Readiness Checklist

Authority Integration Verified

Evidence Integration Verified

Explanation Generation Verified

Recommendation Traceability Verified

Recommendation Reproducibility Verified

23.13 Authority Readiness Checklist

Authority Verification Validated

Confidence Scoring Validated

Authority History Verified

Authority Auditability Verified

23.14 Evidence Readiness Checklist

Evidence Validation Verified

Evidence Versioning Verified

Evidence Lineage Verified

Evidence Auditability Verified

23.15 Security Readiness

Requirements:

Authentication Tested

Authorization Tested

Secrets Protected

TLS Enabled

Security Audit Complete

23.16 Security Validation Checklist

Penetration Testing Complete

Dependency Scanning Complete

Container Scanning Complete

Secret Scanning Complete

Security Approval Granted

23.17 Infrastructure Readiness

Requirements:

Environment Stable

Network Validated

Storage Validated

Observability Enabled

Capacity Verified

23.18 Infrastructure Checklist

Compute Resources Available

Database Capacity Verified

Redis Capacity Verified

Monitoring Operational

Alerting Operational

Backup Operational

23.19 Operational Readiness

Requirements:

Runbooks Complete

Incident Procedures Defined

Escalation Procedures Defined

Support Ownership Defined

Operational Approval Granted

23.20 Operational Checklist

Runbooks Reviewed

Incident Testing Complete

Recovery Testing Complete

Monitoring Review Complete

Operational Training Complete

23.21 Governance Readiness

Requirements:

Auditability Verified

Traceability Verified

Reproducibility Verified

Compliance Verified

Governance Approval Granted

23.22 Governance Checklist

Audit Records Verified

Trace Records Verified

Lineage Verified

Replay Validation Complete

Policy Compliance Verified

23.23 Disaster Recovery Readiness

Requirements:

Backup Validation Complete

Recovery Validation Complete

Recovery Testing Complete

RTO Verified

RPO Verified

23.24 Recovery Checklist

Database Recovery Tested

Infrastructure Recovery Tested

Application Recovery Tested

Security Recovery Tested

Observability Recovery Tested

23.25 Testing Readiness

Requirements:

Unit Tests Passed

Integration Tests Passed

Regression Tests Passed

Security Tests Passed

Performance Tests Passed

23.26 Testing Checklist

Coverage Targets Met

No Critical Defects

No Open Security Issues

No Governance Violations

Release Validation Complete

23.27 Performance Readiness

Validate:

API Latency

Search Latency

Ranking Latency

Recommendation Latency

Database Performance

Cache Performance

23.28 Observability Readiness

Requirements:

Metrics Operational

Logs Operational

Traces Operational

Dashboards Operational

Alerts Operational

23.29 Release Approval Workflow

Engineering Approval



Security Approval



Operations Approval



Governance Approval



Production Approval

23.30 Go-Live Decision Matrix

Status:

NOT_READY

READY_WITH_RISK

READY

Only READY systems may proceed.

23.31 Production Deployment Validation

Validate:

Application Health

API Health

Database Health

Cache Health

Telemetry Health

Security Health

23.32 Post-Go-Live Monitoring

Monitoring Window:

First 24 Hours

First 7 Days

First 30 Days

Focus:

Errors

Latency

Security

Operational Stability

23.33 Rollback Readiness

Must Exist:

Rollback Procedure

Rollback Validation

Rollback Ownership

Rollback Approval Process

23.34 Production Sign-Off Record

Required Information:

Version

Approvers

Approval Date

Readiness Evidence

Deployment Window

Risk Assessment

23.35 Production Auditability

Every go-live generates:

Deployment Event



Approval Record



Audit Record



Trace Record



Release Record

23.36 Production Metrics

Metrics:

Deployment Success Rate

Rollback Rate

Incident Rate

Availability

Error Rate

Performance Compliance

23.37 Governance Rules

READY-GOV-01

Production readiness requires evidence.

READY-GOV-02

Production readiness requires approvals.

READY-GOV-03

Production readiness requires testing.

READY-GOV-04

Production readiness requires security validation.

READY-GOV-05

Production readiness requires recovery validation.

READY-GOV-06

Go-live decisions must be auditable.

23.38 Architectural Constraints

AC-READY-01

Unapproved releases cannot reach production.

AC-READY-02

Security failures block go-live.

AC-READY-03

Governance failures block go-live.

AC-READY-04

Recovery failures block go-live.

AC-READY-05

Critical defects block go-live.

AC-READY-06

Production releases require documented sign-off.

23.39 Section Summary

The TODP Production Readiness Architecture establishes a governed framework ensuring that only validated, secure, recoverable, observable, and compliant systems enter production.

Implementation



Validation



Approval



Deployment



Monitoring



Governance

The architecture ensures operational confidence, security, reliability, auditability, traceability, reproducibility, and long-term platform trust.

IAD Progress

🔗 Section 1 — Repository Structure & Implementation Foundation

🔗 Section 2 — Frontend Implementation Architecture (React)

🔗 Section 3 — Backend Implementation Architecture (FastAPI)

🔗 Section 4 — Database Schema Architecture & PostgreSQL Implementation

🔗 Section 5 — Retrieval Implementation Architecture (BM25 + pgvector + Hybrid Search)

🔗 Section 6 — Ranking Engine Implementation Architecture

- 🔗 Section 7 — Recommendation Engine Implementation Architecture
- 🔗 Section 8 — Authority Verification Implementation Architecture
- 🔗 Section 9 — Evidence Validation Implementation Architecture
- 🔗 Section 10 — Auditability, Traceability & Reproducibility Implementation Architecture
- 🔗 Section 11 — Event-Driven Architecture & Inter-Service Communication Implementation
- 🔗 Section 12 — Authentication & Authorization Implementation Architecture
- 🔗 Section 13 — API Gateway & External Interface Implementation Architecture
- 🔗 Section 14 — Redis Caching Architecture & Performance Optimization Implementation
- 🔗 Section 15 — Observability Implementation Architecture
- 🔗 Section 16 — Testing & Quality Assurance Implementation Architecture
- 🔗 Section 17 — CI/CD & Deployment Implementation Architecture
- 🔗 Section 18 — Infrastructure as Code & Environment Provisioning Architecture
- 🔗 Section 19 — Security Implementation Architecture
- 🔗 Section 20 — Disaster Recovery & Backup Implementation Architecture
- 🔗 Section 21 — Operational Runbooks & Platform Administration Architecture
- 🔗 Section 22 — Development Standards & Coding Guidelines Architecture
- 🔗 Section 23 — Production Readiness Checklist & Go-Live Architecture
- 🔗 Section 24 — Implementation Governance & Change Management Architecture

IAD V1 — Section 24

Implementation Governance & Change Management Architecture

Document: Implementation Architecture Document (IAD V1)

Section: 24

Status: APPROVED

Version: V1

Parent Documents:

- PRD V1 (Frozen)
- ADD V1 (Frozen)
- DDD V1 (Frozen)
- SDD V1 (Frozen)

24.1 Purpose

This section defines the governance and change management architecture governing implementation activities within TODP.

The objective is to ensure:

- Controlled evolution
- Architectural integrity
- Traceable decision making
- Auditable modifications
- Reproducible releases
- Long-term platform sustainability

All implementation changes must follow governed processes.

24.2 Governance Philosophy

Without governance:

Architecture degrades.

Consistency declines.

Complexity increases.

Trust decreases.

Governance exists to protect platform integrity.

24.3 Governance Principles

GOV-01

Changes require justification.

GOV-02

Changes require review.

GOV-03

Changes require approval.

GOV-04

Changes require traceability.

GOV-05

Changes require auditability.

GOV-06

Changes require reproducibility.

24.4 Governance Architecture

Change Request



Review



Impact Analysis



Approval



Implementation



Validation



Audit



Historical Preservation

24.5 Governance Scope

Governed Assets:

Architecture

Code

Database Schema

Infrastructure

Security Controls

Policies

Operational Procedures

Documentation

24.6 Governance Authorities

Authority Level 1

Engineering Governance

Authority Level 2

Security Governance

Authority Level 3

Operational Governance

Authority Level 4

Executive Governance

24.7 Change Categories

Category A

Standard Change

Category B

Significant Change

Category C

Critical Change

Category D

Emergency Change

24.8 Standard Changes

Examples:

Bug Fixes

Documentation Updates

Minor Refactoring

Configuration Updates

Low-Risk Improvements

24.9 Significant Changes

Examples:

Major Features

Schema Changes

API Changes

Infrastructure Changes

Security Enhancements

24.10 Critical Changes

Examples:

Authentication Changes

Authorization Changes

Governance Changes

Core Ranking Logic Changes

Audit Architecture Changes

24.11 Emergency Changes

Examples:

Security Incident Response

Critical Production Failure

Major Outage Recovery

Regulatory Compliance Actions

24.12 Change Request Structure

Required Fields:

Change ID

Title

Description

Business Justification

Risk Assessment

Affected Components

Owner

Approval Status

24.13 Change Lifecycle

Proposed

↓

Reviewed

↓

Approved



Implemented



Validated



Released



Archived

24.14 Impact Analysis

Every significant change requires:

Technical Impact

Operational Impact

Security Impact

Governance Impact

Performance Impact

Recovery Impact

24.15 Architecture Review

Required For:

Database Changes

API Changes

Infrastructure Changes

Security Changes

Core Platform Changes

24.16 Security Review

Required For:

Authentication

Authorization

Secrets

Infrastructure Security

Compliance Controls

24.17 Governance Review

Required For:

Audit Controls

Traceability Controls

Reproducibility Controls

Policy Changes

Decision Framework Changes

24.18 Approval Matrix

Standard Change

Engineering Approval

Significant Change

Engineering + Operations

Critical Change

Engineering + Security + Governance

Emergency Change

Emergency Approval Process

Followed By Review

24.19 Change Documentation

Required Documentation:

Design Updates

Implementation Notes

Testing Evidence

Release Notes

Runbook Updates

24.20 Change Traceability

Every change receives:

change_id

correlation_id

approval_record

audit_record

deployment_record

24.21 Change Auditability

Every change generates:

Change Event



Audit Record



Trace Record



Governance Record

24.22 Version Management

Managed Assets:

Code Versions

API Versions

Schema Versions

Infrastructure Versions

Policy Versions

Documentation Versions

24.23 Configuration Governance

Changes To:

Environment Variables

Security Settings

Infrastructure Settings

Feature Flags

Monitoring Settings

Require governance controls.

24.24 Database Governance

Schema changes require:

Migration

Testing

Rollback Validation

Review

Approval

Direct production schema changes prohibited.

24.25 API Governance

Changes require:

Contract Validation

Backward Compatibility Analysis

Version Review

Documentation Update

24.26 Infrastructure Governance

Changes require:

IaC Update

Review

Validation

Deployment Approval

Audit Record

24.27 Security Governance

Changes require:

Security Review

Threat Assessment

Validation

Approval

Audit Record

24.28 Operational Governance

Changes require:

Runbook Updates

Support Validation

Operational Approval

Recovery Validation

24.29 Release Governance

Every release requires:

Approved Changes

Validated Testing

Security Approval

Operational Approval

Governance Approval

24.30 Emergency Governance

Emergency Process:

Identify Issue



Emergency Approval



Implement Fix



Validate Fix



Post-Implementation Review

24.31 Governance Repository Structure

docs/governance/

└─ policies/

└─ decisions/

└─ approvals/

└─ change_requests/

└─ architecture_reviews/

└─ release_records/

24.32 Governance Metrics

Metrics:

Change Volume

Approval Time

Deployment Success Rate

Rollback Rate

Emergency Changes

Policy Violations

24.33 Governance Reporting

Reports:

Weekly Change Report

Monthly Governance Report

Quarterly Architecture Review

Security Governance Report

Operational Governance Report

24.34 Compliance Validation

Validate:

Architecture Compliance

Security Compliance

Operational Compliance

Governance Compliance

Documentation Compliance

24.35 Governance Testing

Validate:

Approval Workflows

Audit Generation

Trace Generation

Policy Enforcement

Change Lifecycle

24.36 Governance Observability

Monitor:

Approval Activity

Policy Violations

Change Activity

Release Activity

Audit Activity

Trace Activity

24.37 Governance Rules

GOV-GOV-01

Significant changes require review.

GOV-GOV-02

Critical changes require approval.

GOV-GOV-03

Changes require audit records.

GOV-GOV-04

Changes require traceability.

GOV-GOV-05

Governance records must be preserved.

GOV-GOV-06

Emergency changes require retrospective review.

24.38 Architectural Constraints

AC-GOV-01

Unapproved critical changes are prohibited.

AC-GOV-02

Direct production modifications are prohibited.

AC-GOV-03

Governance records cannot be deleted.

AC-GOV-04

Critical changes require impact analysis.

AC-GOV-05

Release governance cannot be bypassed.

AC-GOV-06

Architecture governance remains mandatory.

24.39 Section Summary

The TODP Governance Architecture establishes a controlled framework for managing platform evolution while preserving trust, consistency, and architectural integrity.

Change Request

↓

Review



Approval



Implementation



Validation



Release



Audit



Historical Preservation

The architecture ensures that all platform changes remain governed, auditable, traceable, reproducible, and aligned with TODP's foundational principles.

IAD Progress

- 📄 Section 1 — Repository Structure & Implementation Foundation
- 📄 Section 2 — Frontend Implementation Architecture (React)
- 📄 Section 3 — Backend Implementation Architecture (FastAPI)
- 📄 Section 4 — Database Schema Architecture & PostgreSQL Implementation
- 📄 Section 5 — Retrieval Implementation Architecture (BM25 + pgvector + Hybrid Search)
- 📄 Section 6 — Ranking Engine Implementation Architecture
- 📄 Section 7 — Recommendation Engine Implementation Architecture
- 📄 Section 8 — Authority Verification Implementation Architecture
- 📄 Section 9 — Evidence Validation Implementation Architecture
- 📄 Section 10 — Auditability, Traceability & Reproducibility Implementation Architecture
- 📄 Section 11 — Event-Driven Architecture & Inter-Service Communication Implementation
- 📄 Section 12 — Authentication & Authorization Implementation Architecture
- 📄 Section 13 — API Gateway & External Interface Implementation Architecture
- 📄 Section 14 — Redis Caching Architecture & Performance Optimization Implementation
- 📄 Section 15 — Observability Implementation Architecture
- 📄 Section 16 — Testing & Quality Assurance Implementation Architecture
- 📄 Section 17 — CI/CD & Deployment Implementation Architecture
- 📄 Section 18 — Infrastructure as Code & Environment Provisioning Architecture
- 📄 Section 19 — Security Implementation Architecture
- 📄 Section 20 — Disaster Recovery & Backup Implementation Architecture
- 📄 Section 21 — Operational Runbooks & Platform Administration Architecture
- 📄 Section 22 — Development Standards & Coding Guidelines Architecture
- 📄 Section 23 — Production Readiness Checklist & Go-Live Architecture
- 📄 Section 24 — Implementation Governance & Change Management Architecture

▣ Section 25 — IAD V1 Conclusion & Approved Implementation Baseline

IAD V1 — Section 25

IAD V1 Conclusion & Approved Implementation Baseline

Document: Implementation Architecture Document (IAD V1)

Section: 25

Status: APPROVED

Version: V1 FINAL

Parent Documents:

- PRD V1 (Frozen)

- ADD V1 (Frozen)

- DDD V1 (Frozen)

- SDD V1 (Frozen)

25.1 Purpose

This section formally concludes IAD V1 and establishes the approved implementation baseline for TODP.

The purpose of IAD V1 is to translate approved architecture and design decisions into an implementable engineering blueprint.

This document defines how TODP shall be built.

25.2 Document Position Within TODP

TODP Documentation Hierarchy

PRD V1



ADD V1



DDD V1



SDD V1



IAD V1



Implementation



Testing



Deployment



Operations

25.3 Implementation Authority

IAD V1 is the authoritative implementation reference for:

Frontend Development

Backend Development

Database Development

Infrastructure Engineering

Security Engineering

DevOps Engineering

Platform Operations

Governance Operations

25.4 Scope of IAD V1

IAD V1 defines implementation standards for:

Repository Structure

Application Architecture

Database Architecture

Retrieval Architecture

Ranking Architecture

Recommendation Architecture

Authority Architecture

Evidence Architecture

Governance Architecture

Security Architecture

Operations Architecture

Deployment Architecture

Observability Architecture

Recovery Architecture

25.5 Technology Baseline

Frontend

React

TypeScript

Backend

FastAPI

Python

Database

PostgreSQL

pgvector

Cache

Redis

Observability

OpenTelemetry

Prometheus

Grafana

Infrastructure

Containerized Deployment

Infrastructure as Code

25.6 Retrieval Baseline

Retrieval Strategy:

Hybrid Retrieval

Components:

BM25

+

Vector Search

+

Fusion Layer

Purpose:

Maximize discovery quality.

25.7 Ranking Baseline

Ranking Formula:

Total Cost

=

Money Cost

+

Time Cost

+

Distance Cost

Lowest Total Cost ranks highest.

No sponsored ranking.

No hidden ranking logic.

No paid placement.

25.8 Recommendation Baseline

Recommendation Flow:

Evidence

↓

Authority

↓

Ranking



Recommendation

Recommendations are derived from rankings.

Recommendations do not override rankings.

25.9 Authority Baseline

Authority Requirements:

Verification

Traceability

Auditability

Confidence Assessment

Lifecycle Management

AI cannot act as authority.

25.10 Evidence Baseline

Evidence Requirements:

Validation

Versioning

Lineage

Traceability

Auditability

Historical Preservation

25.11 Security Baseline

Core Security Controls:

Authentication

Authorization

Encryption

Secret Management

Audit Logging

Monitoring

Incident Response

25.12 Governance Baseline

Governance Controls:

Approval Workflows

Audit Records

Trace Records

Change Management

Version Management

Policy Enforcement

25.13 Observability Baseline

Observability Components:

Metrics

Logs

Traces

Dashboards

Alerting

Operational Analytics

Every critical workflow must be observable.

25.14 Auditability Baseline

Every critical workflow generates:

Audit Record



Trace Record



Correlation ID



Historical Preservation

25.15 Traceability Baseline

Every critical platform decision must support:

Source Tracking

Relationship Tracking

Decision Tracking

Lineage Reconstruction

25.16 Reproducibility Baseline

Platform outcomes must support:

Historical Replay

Version Preservation

Snapshot Preservation

Output Reconstruction

Validation

25.17 Operational Baseline

Required Operational Capabilities:

Monitoring

Incident Management

Change Management

Recovery Management

Capacity Management

Runbook Execution

25.18 Recovery Baseline

Required Recovery Capabilities:

Backup

Restoration

Recovery Testing

RTO Validation

RPO Validation

Disaster Recovery

25.19 Deployment Baseline

Deployment Requirements:

CI/CD

Automated Validation

Security Validation

Governance Validation

Production Approval

Release Traceability

25.20 Development Baseline

Development Requirements:

Code Review

Testing

Documentation

Security Validation

Observability

Governance Compliance

25.21 Quality Baseline

Required Validation:

Unit Tests

Integration Tests

Contract Tests

Security Tests

Governance Tests

Performance Tests

25.22 Architectural Constraints

The following constraints are mandatory:

No Sponsored Content

No Paid Placement

No Hidden Ranking Logic

AI Is Not An Authority

Authority Before Recommendation

Evidence First

Traceability

Auditability

Reproducibility

Governance Enforcement

25.23 Implementation Readiness

Upon approval of IAD V1:

The platform possesses:

Approved Product Requirements

Approved Architecture

Approved Domain Design

Approved System Design

Approved Implementation Architecture

The project is implementation ready.

25.24 Implementation Deliverables

Expected Deliverables:

Source Code

Infrastructure Code

Database Schema

API Layer

Retrieval Layer

Ranking Layer

Recommendation Layer

Governance Layer

Observability Layer

Operational Assets

25.25 Version Governance

IAD V1 is frozen after approval.

Future modifications require:

Change Request

Impact Analysis

Governance Review

Version Increment

Approval

25.26 Success Criteria

TODP implementation is considered successful when:

All required capabilities exist.

All governance controls operate.

All security controls operate.

All observability controls operate.

All recovery controls operate.

All architectural principles remain intact.

25.27 Approved Implementation Baseline

The following baseline is approved:

Architecture:

Approved

Design:

Approved

Technology Stack:

Approved

Security Model:

Approved

Governance Model:

Approved

Operational Model:

Approved

Implementation Standards:

Approved

25.28 Final Architectural Statement

TODP is designed as a transparent, evidence-driven opportunity discovery platform.

The platform is governed by:

Evidence First



Authority Before Recommendation



Ranking Before Recommendation



Auditability



Traceability



Reproducibility



Governance

The implementation architecture exists to preserve these principles throughout the lifecycle of the platform.

25.29 Final Approval Statement

This document establishes the official implementation baseline for TODP Version 1.

All implementation activities shall align with:

PRD V1

ADD V1

DDD V1

SDD V1

IAD V1

Any deviation requires formal governance review and approval.

25.30 IAD V1 Completion Declaration

Implementation Architecture Document

Version: V1

Status: COMPLETE

Approval Status: READY FOR FREEZE

Document State:

FINAL

APPROVED

IMPLEMENTATION BASELINE ESTABLISHED

IAD V1 Progress

- 📄 Section 1 — Repository Structure & Implementation Foundation
- 📄 Section 2 — Frontend Implementation Architecture (React)
- 📄 Section 3 — Backend Implementation Architecture (FastAPI)
- 📄 Section 4 — Database Schema Architecture & PostgreSQL Implementation
- 📄 Section 5 — Retrieval Implementation Architecture (BM25 + pgvector + Hybrid Search)
- 📄 Section 6 — Ranking Engine Implementation Architecture
- 📄 Section 7 — Recommendation Engine Implementation Architecture
- 📄 Section 8 — Authority Verification Implementation Architecture
- 📄 Section 9 — Evidence Validation Implementation Architecture
- 📄 Section 10 — Auditability, Traceability & Reproducibility Implementation Architecture
- 📄 Section 11 — Event-Driven Architecture & Inter-Service Communication Implementation
- 📄 Section 12 — Authentication & Authorization Implementation Architecture
- 📄 Section 13 — API Gateway & External Interface Implementation Architecture
- 📄 Section 14 — Redis Caching Architecture & Performance Optimization Implementation
- 📄 Section 15 — Observability Implementation Architecture
- 📄 Section 16 — Testing & Quality Assurance Implementation Architecture
- 📄 Section 17 — CI/CD & Deployment Implementation Architecture
- 📄 Section 18 — Infrastructure as Code & Environment Provisioning Architecture
- 📄 Section 19 — Security Implementation Architecture
- 📄 Section 20 — Disaster Recovery & Backup Implementation Architecture
- 📄 Section 21 — Operational Runbooks & Platform Administration Architecture
- 📄 Section 22 — Development Standards & Coding Guidelines Architecture
- 📄 Section 23 — Production Readiness Checklist & Go-Live Architecture
- 📄 Section 24 — Implementation Governance & Change Management Architecture
- 📄 Section 25 — IAD V1 Conclusion & Approved Implementation Baseline

IAD V1 STATUS: COMPLETE

IAD V1 STATUS: READY FOR FREEZE

IAD V1 STATUS: IMPLEMENTATION READY
